Review pack: AgentCore Runtime × Redshift × IAM Identity Center (OBO / 2LO / 3LO)

Generated 2026-10-05 16:38 HKT · account `383386985941` · 43 resources · 40 console screenshots · verification **29 PASS / 0 FAIL**

All resources were deployed by the numbered, idempotent scripts in `infra/` (01 → 13), in this order. For every step this pack lists what the script created, an AWS console screenshot per resource (exact URL and capture time shown under each image; open the URL to validate), and read-only verification commands run when the pack was built, with the expected pattern and the actual output. Functional test results (OBO 16/16 matrix, 2LO/3LO 18/18) are in `RUNBOOK.md` and `tests/results-*.txt`.

Summary (deployment order)

Step	What was deployed	Resources	Screenshots	Checks
1	Runtime VPC with a fixed egress IP (ap-southeast-1)	10	10	**3 PASS**
2	Cognito user pool (end-user IdP, admin-only users)	3	3	**2 PASS**
3	IdC Trusted Token Issuer + Redshift IdC application	4	4	**2 PASS**
4	Redshift cluster, network-restricted (ap-east-1)	4	3	**2 PASS**
5	Seed data + grants to IdC group roles (SQL)	2	0	**2 PASS**
6	OBO runtime execution role (no Redshift permissions)	1	1	**1 PASS**
7	IdC customer-managed app for the agent's token exchange	1	1	**3 PASS**
8	OBO data agent on AgentCore Runtime	2	2	**2 PASS**
9	Cognito OAuth clients for 2LO / 3LO (same pool)	4	5	**3 PASS**
10	AgentCore Identity credential providers	2	3	**2 PASS**
11	2LO downstream: Gateway → Lambda → Redshift service user	6	5	**3 PASS**
12	Separate OAuth agent runtime (2LO / 3LO)	3	2	**3 PASS**
13	Browser sign-in for the local web GUIs	1	1	**1 PASS**

Step 1: Runtime VPC with a fixed egress IP (ap-southeast-1)

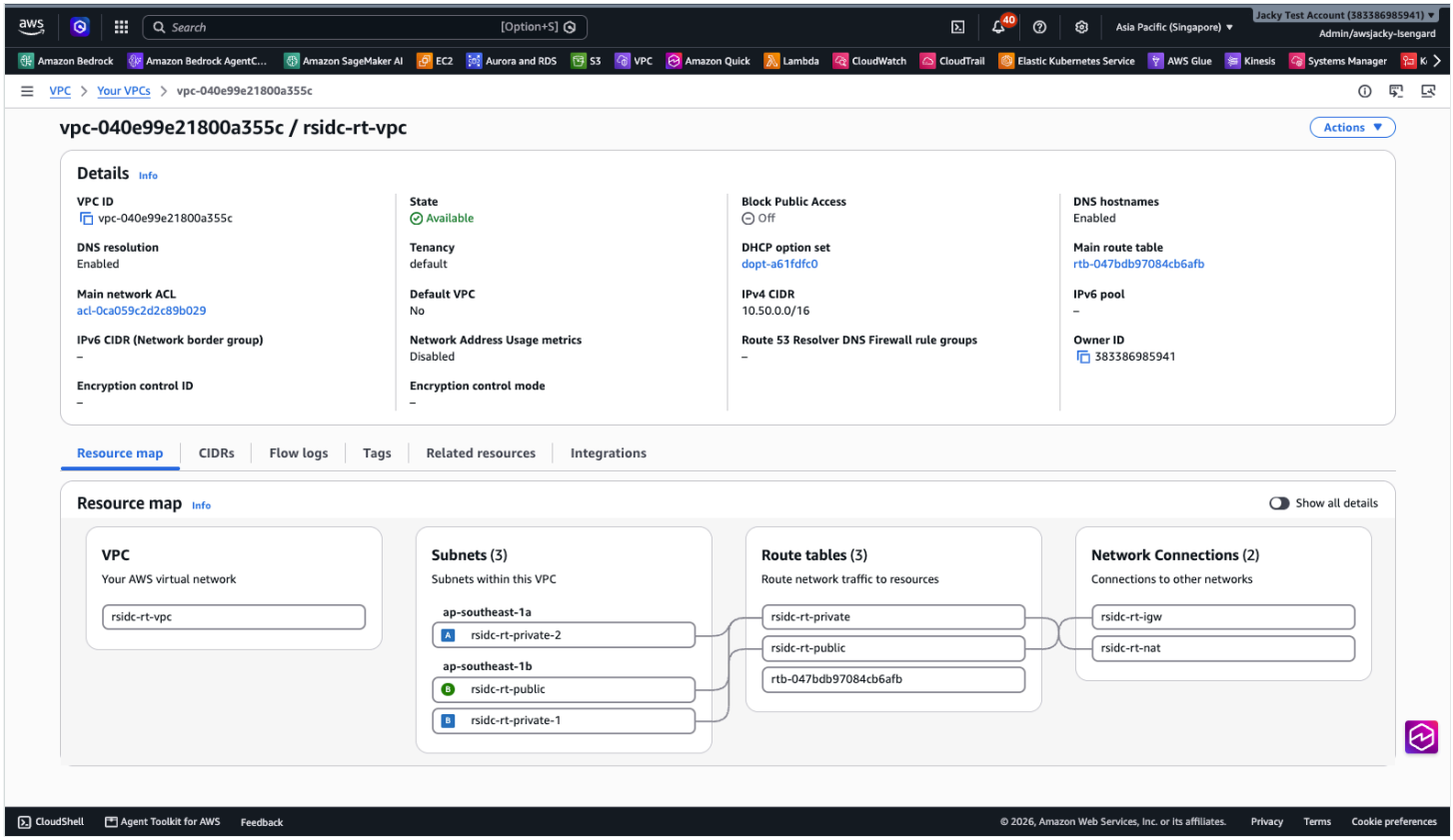
Private subnets for the AgentCore Runtime ENIs; all egress goes through one NAT gateway with an Elastic IP, so Redshift can allowlist exactly one IP for the agent.

IaC / script: `infra/01-runtime-network.sh`
Deploy command: `infra/run.sh ./infra/01-runtime-network.sh`

Deployed resources

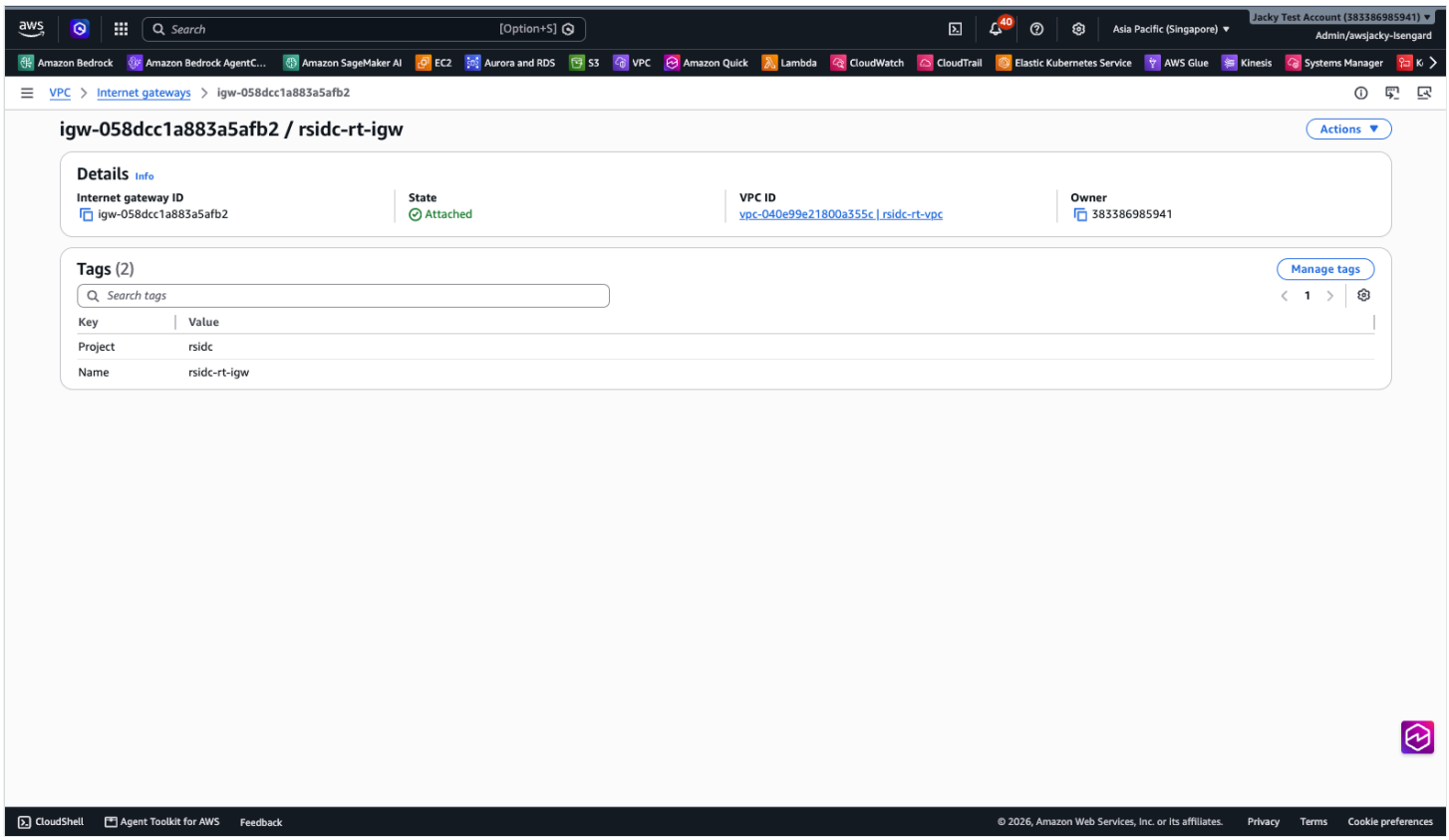
Type	ID / ARN	Name
AWS::EC2::VPC	`vpc-040e99e21800a355c`	rsidc-rt-vpc 10.50.0.0/16
AWS::EC2::InternetGateway	`igw-058dcc1a883a5afb2`	rsidc-rt-igw
AWS::EC2::Subnet	`subnet-0fd4195de9bdffba9`	public 10.50.0.0/24
AWS::EC2::Subnet	`subnet-0bf130ed318f05d66`	private 10.50.1.0/24
AWS::EC2::Subnet	`subnet-0a3ac30526d744713`	private 10.50.2.0/24
AWS::EC2::RouteTable	`rtb-0f37a8edf400fa73c`	public → IGW
AWS::EC2::RouteTable	`rtb-0316eeecb91a0b588`	private → NAT
AWS::EC2::EIP	`eipalloc-08e9ba07612e963ca`	47.131.152.68
AWS::EC2::NatGateway	`nat-042a3797afe93a19f`	rsidc-rt-nat
AWS::EC2::SecurityGroup	`sg-03bd7cebf86c77315`	rsidc-runtime-sg (egress only)

Console: VPC rsidc-rt-vpc 10.50.0.0/16 (runtime network)
URL at capture (2026-10-05 16:13:03 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#VpcDetails:VpcId=vpc-040e99e21800a355c



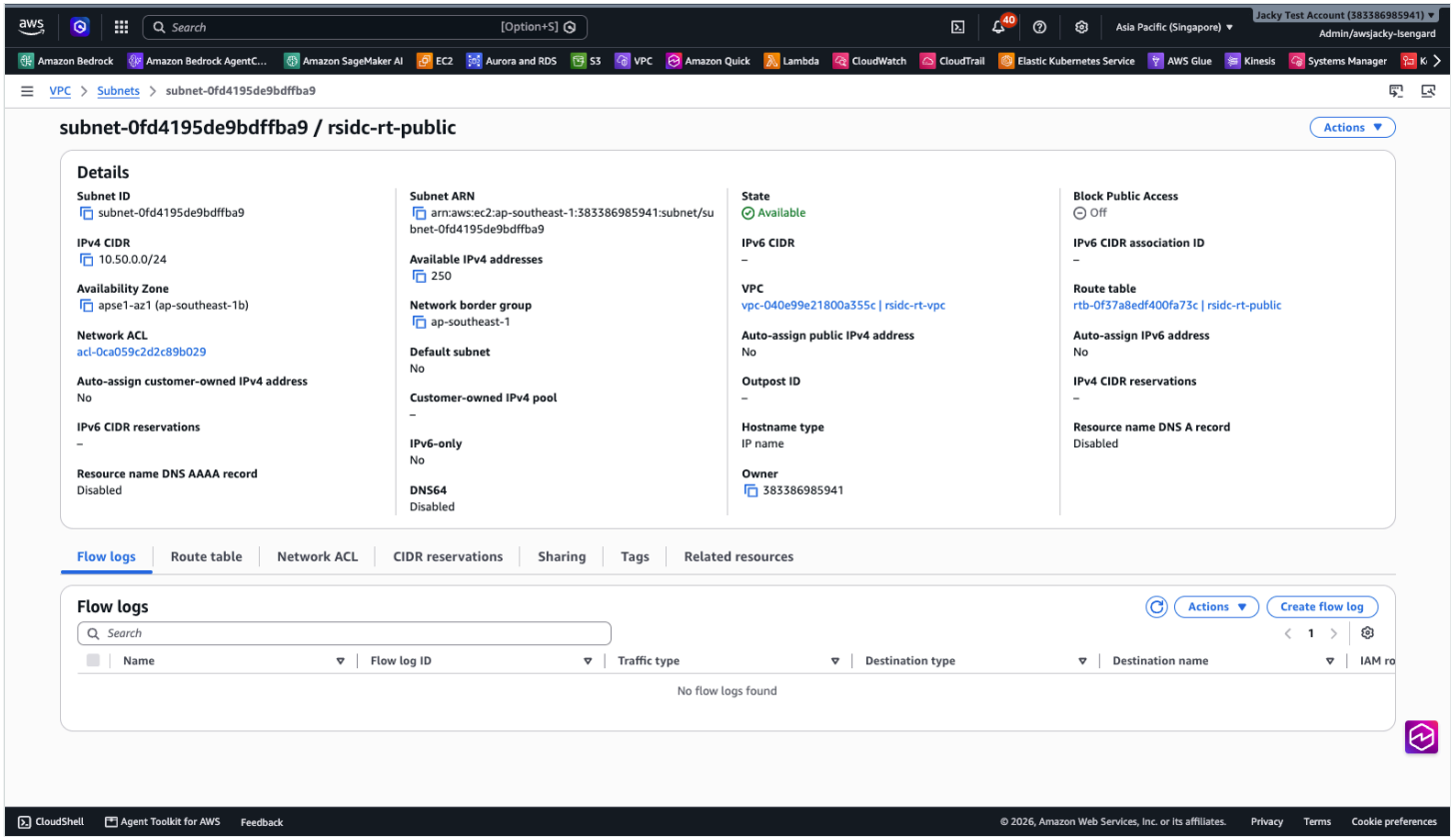
Console: Internet gateway attached to rsidc-rt-vpc

URL at capture (2026-10-05 16:13:10 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#InternetGateway:internetGatewayId=igw-058dcc1a883a5afb2



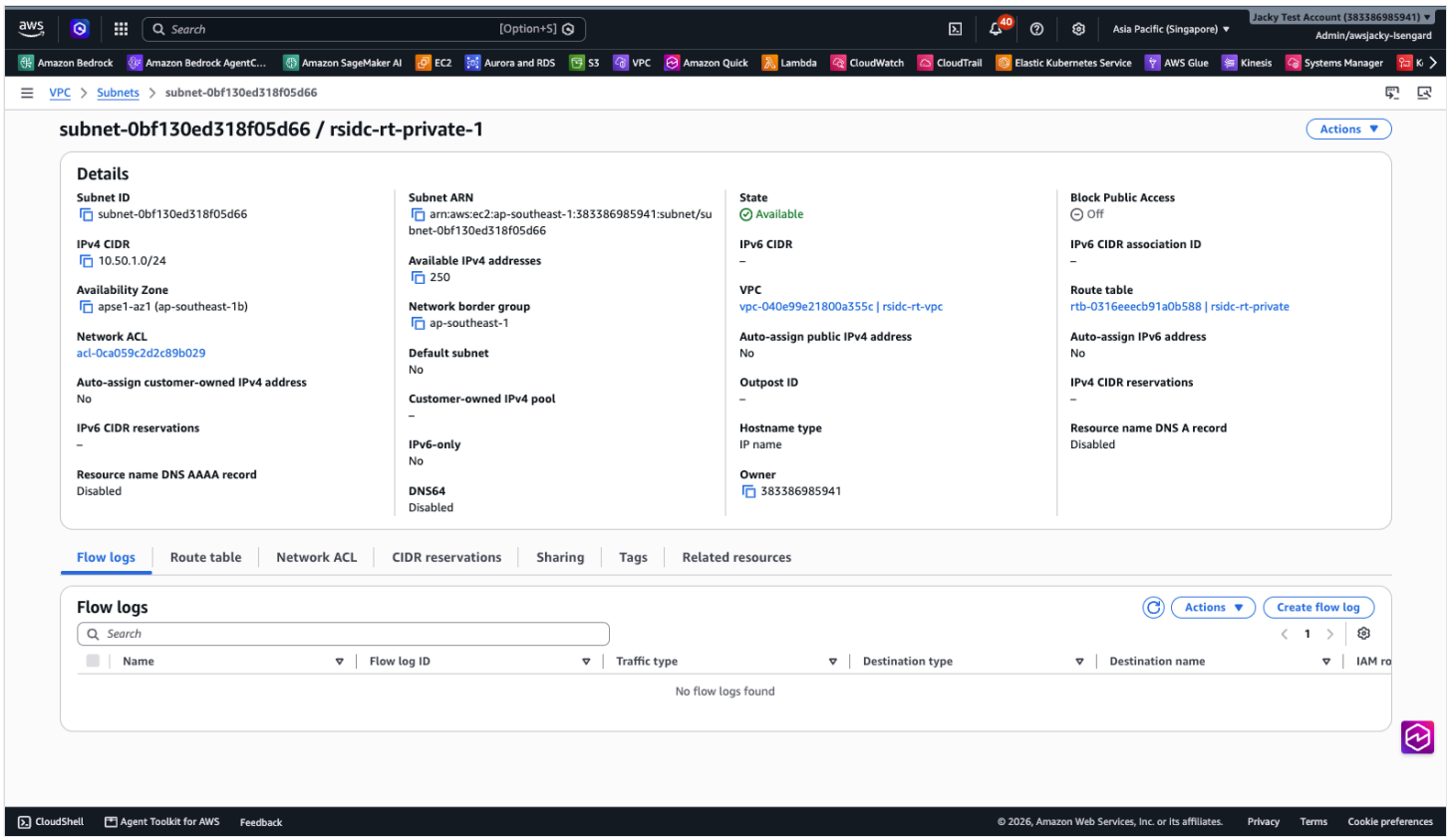
Console: Public subnet 10.50.0.0/24 (apse1-az1) for the NAT gateway

URL at capture (2026-10-05 16:13:17 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#SubnetDetails:subnetId=subnet-0fd4195de9bdffba9



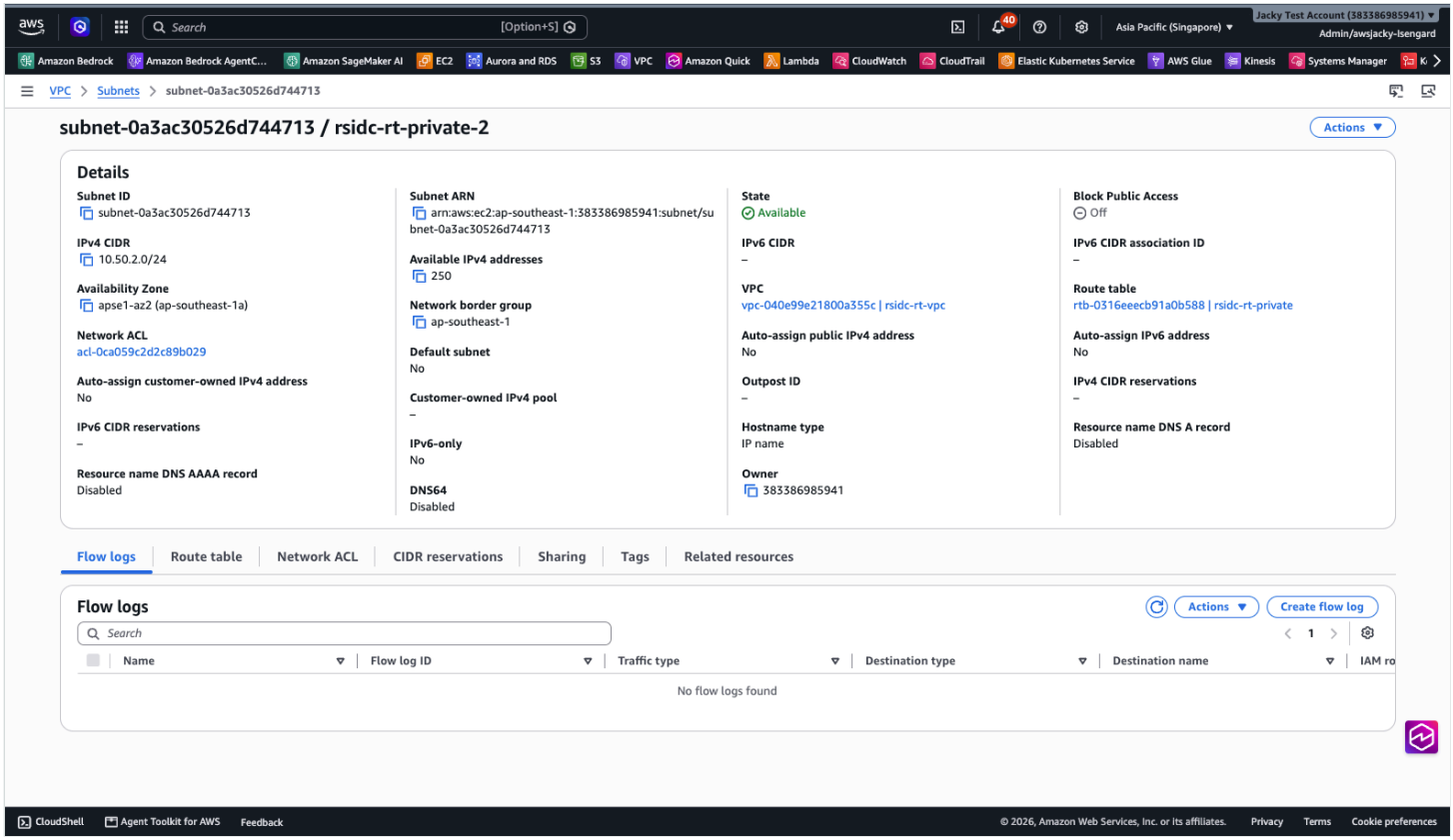
Console: Private subnet 1 10.50.1.0/24 (apse1-az1) for runtime ENIs

URL at capture (2026-10-05 16:13:25 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#SubnetDetails:subnetId=subnet-0bf130ed318f05d66



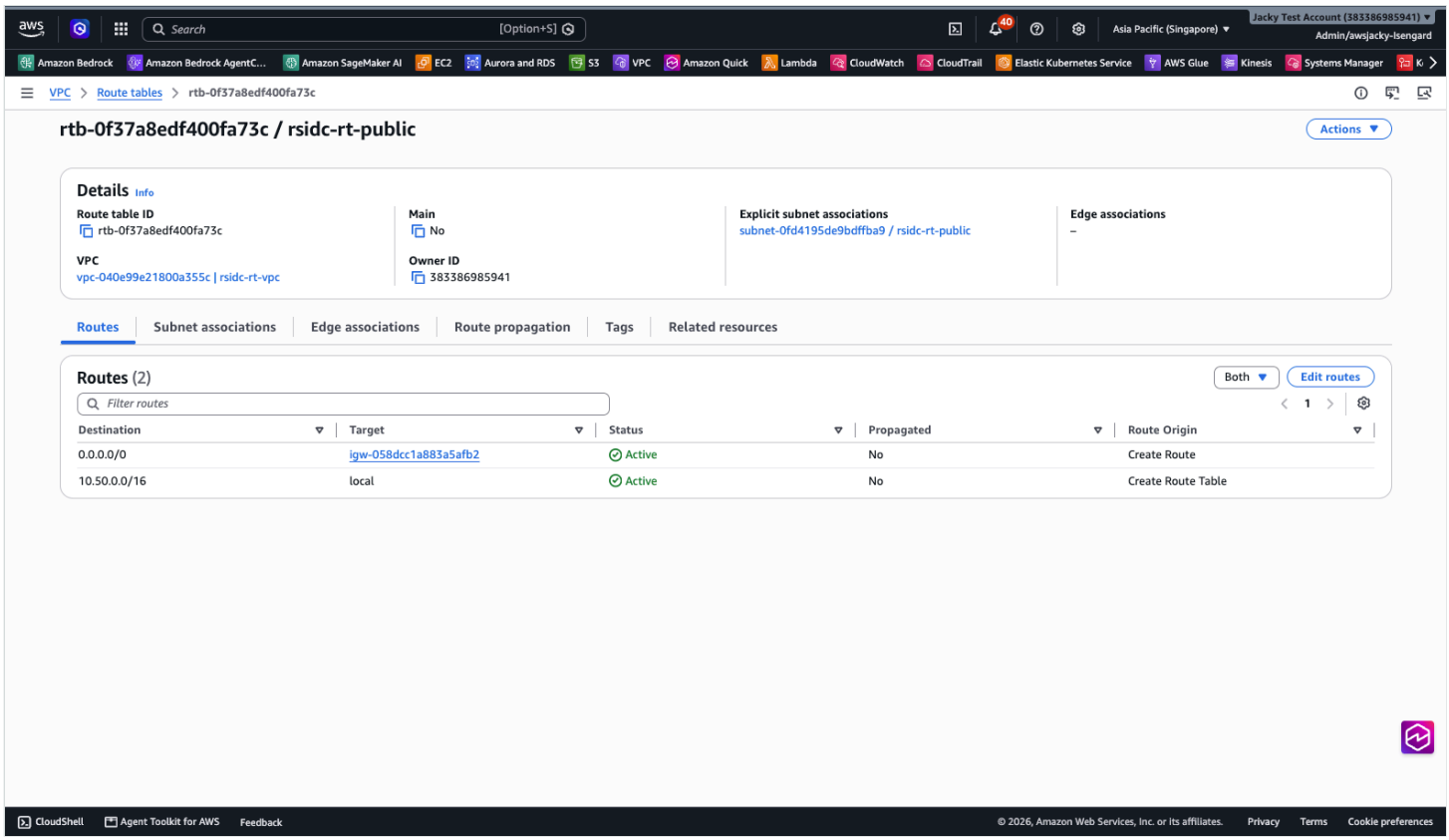
Console: Private subnet 2 10.50.2.0/24 (apse1-az2) for runtime ENIs

URL at capture (2026-10-05 16:13:32 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#SubnetDetails:subnetId=subnet-0a3ac30526d744713



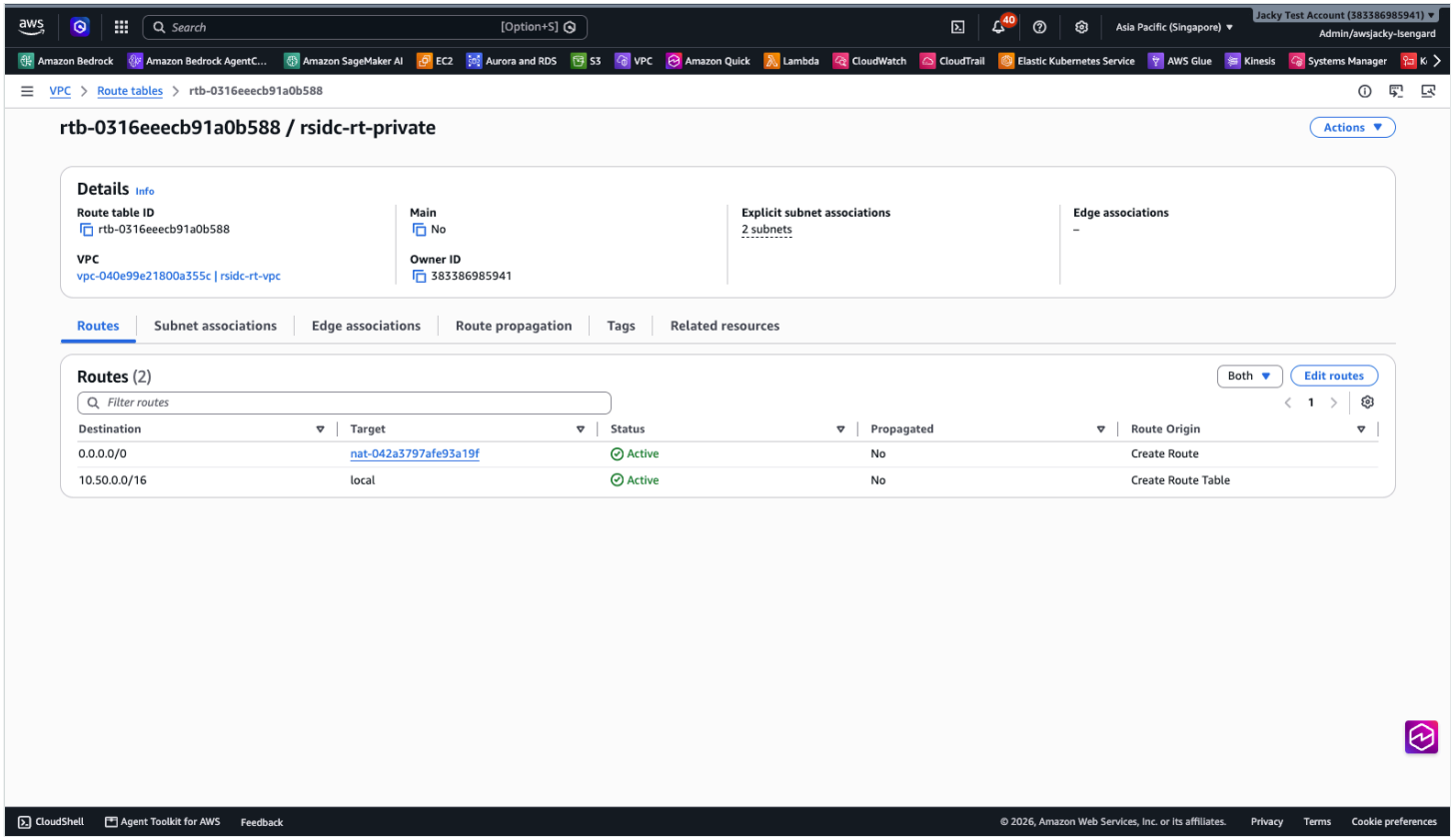
Console: Public route table: 0.0.0.0/0 → internet gateway

URL at capture (2026-10-05 16:13:44 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#RouteTableDetails:RouteTableId=rtb-0f37a8edf400fa73c



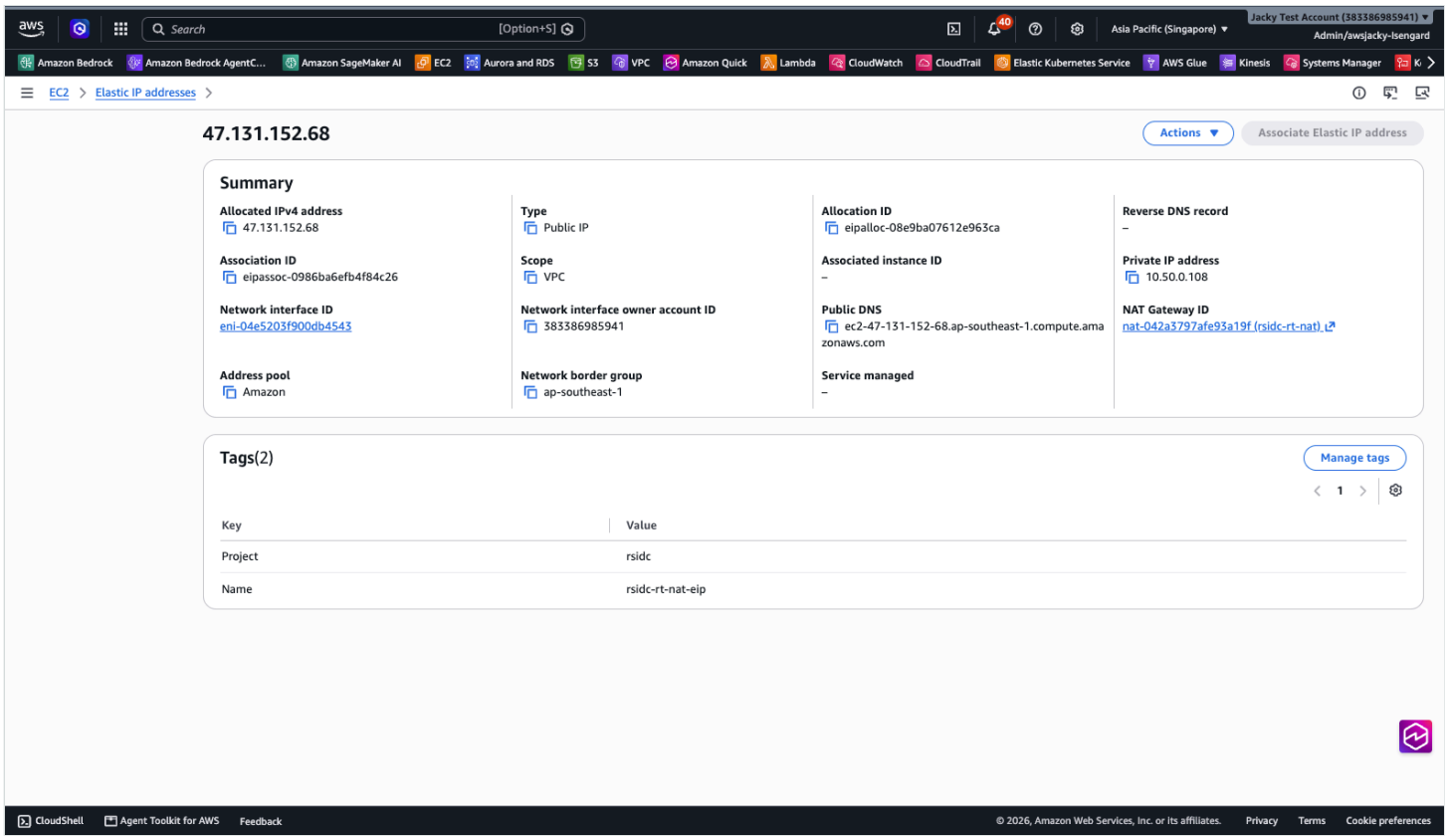
Console: Private route table: 0.0.0.0/0 → NAT gateway

URL at capture (2026-10-05 16:13:56 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#RouteTableDetails:RouteTableId=rtb-0316eeecb91a0b588



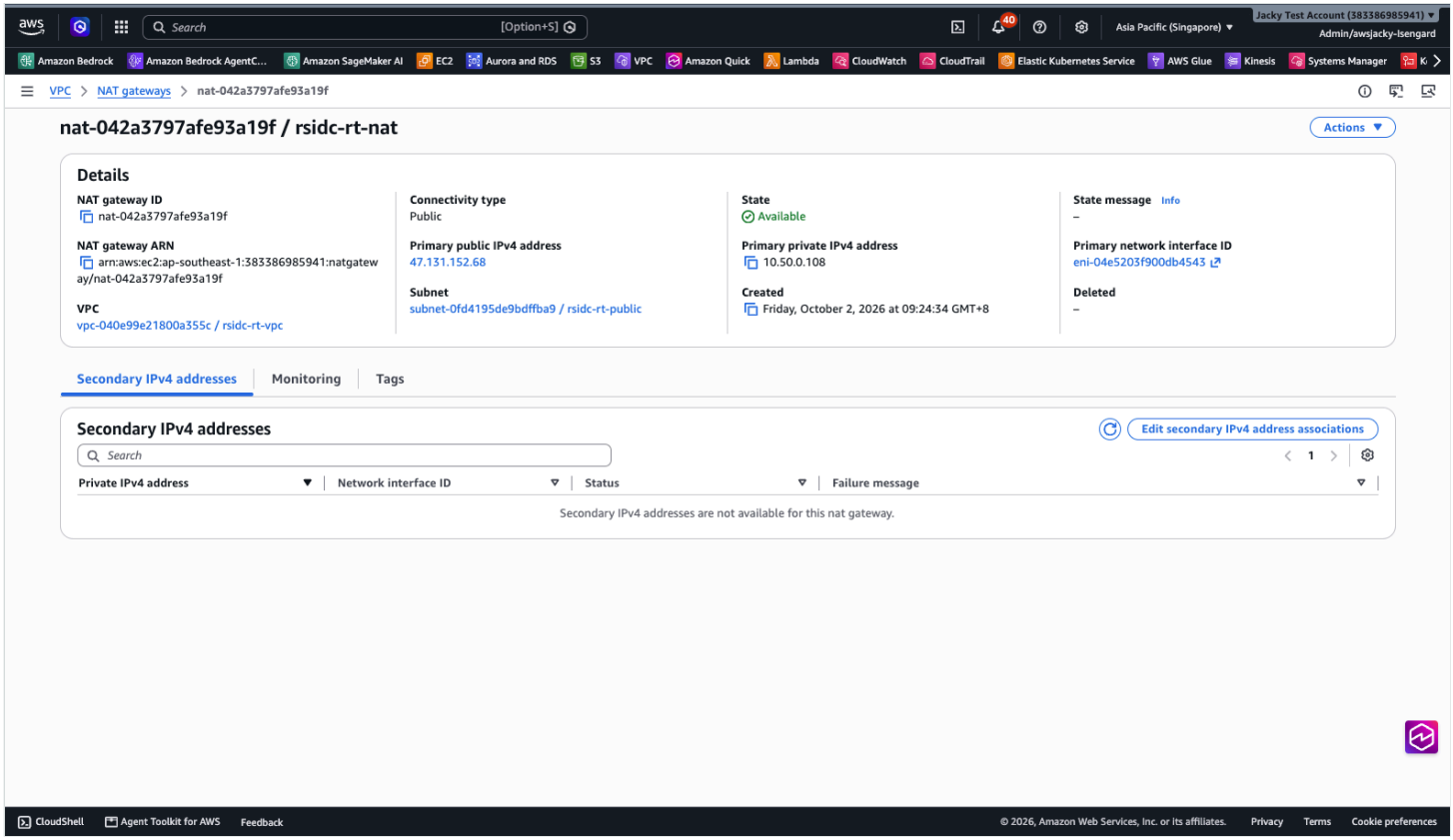
Console: Elastic IP 47.131.152.68 (the runtime's only egress IP)

URL at capture (2026-10-05 16:14:06 HKT): https://ap-southeast-1.console.aws.amazon.com/ec2/home?region=ap-southeast-1#ElasticIpDetails:AllocationId=eipalloc-08e9ba07612e963ca



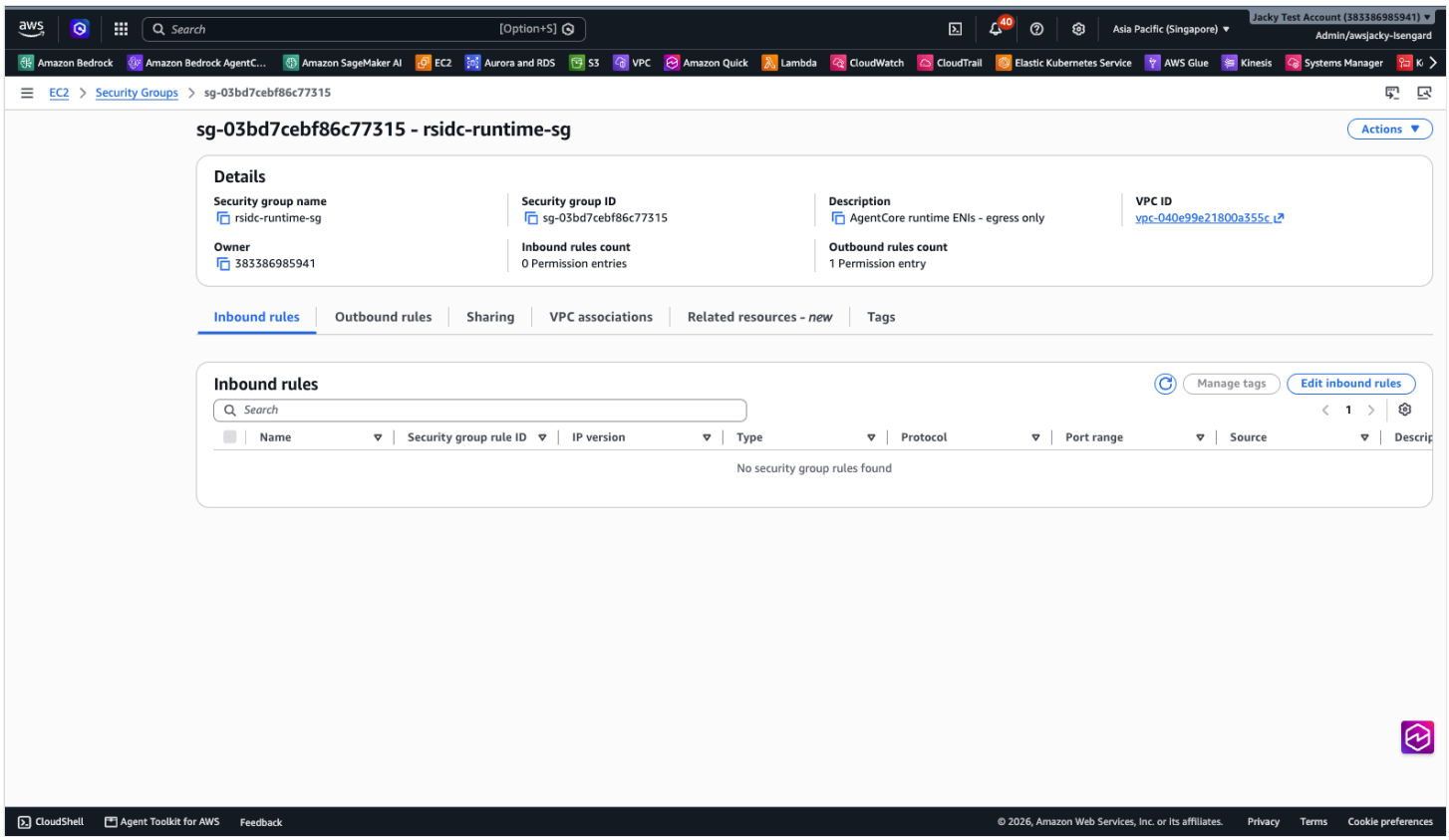
Console: NAT gateway rsidc-rt-nat (public, EIP 47.131.152.68)

URL at capture (2026-10-05 16:14:14 HKT): https://ap-southeast-1.console.aws.amazon.com/vpcconsole/home?region=ap-southeast-1#NatGatewayDetails:natGatewayId=nat-042a3797afe93a19f



Console: Runtime security group: no inbound rules (egress only)

URL at capture (2026-10-05 16:14:22 HKT): https://ap-southeast-1.console.aws.amazon.com/ec2/home?region=ap-southeast-1#SecurityGroup:groupId=sg-03bd7cebf86c77315



Verification

Check	Command	Expected	Result
NAT available with the EIP	`aws ec2 describe-nat-gateways --region $RT_REGION --nat-gateway-ids $RT_NAT_ID --query 'NatGateways[0].[State,NatGatewayAddresses[0].PublicIp]' --output text`	`available\s+47\.131\.152\.68`	PASS
private subnets route to NAT	`aws ec2 describe-route-tables --region $RT_REGION --route-table-ids $RT_PRIV_RTB --query 'RouteTables[0].Routes[?DestinationCidrBlock==`0.0.0.0/0`].NatGatewayId' --output text`	`nat-042a3797afe93a19f`	PASS

runtime SG has no inbound rules	`aws ec2 describe-security-groups --region $RT_REGION --group-ids $RT_SG_ID --query 'length(SecurityGroups[0].IpPermissions)'`	`^0$`	**PASS**

▼ Actual output: NAT available with the EIP

```
available       47.131.152.68
```

▼ Actual output: private subnets route to NAT

```
nat-042a3797afe93a19f
```

▼ Actual output: runtime SG has no inbound rules

```
0
```

Step 2: Cognito user pool (end-user IdP, admin-only users)

Cognito issues the users' ID tokens (JWT). Users are admin-created with emails that mirror IAM Identity Center users; self sign-up is disabled because email is the identity mapping key.

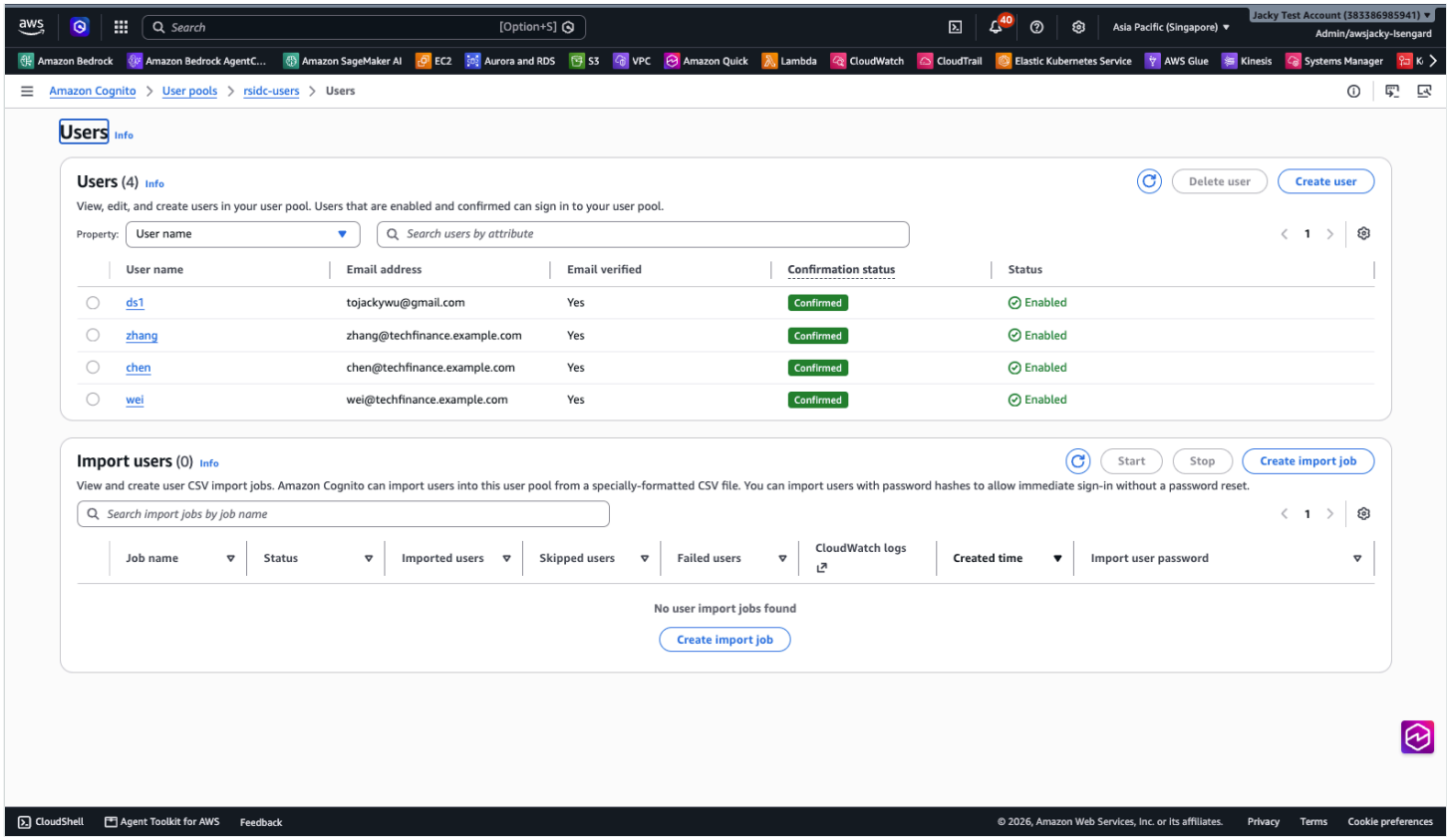
IaC / script: `infra/02-cognito.sh`
Deploy command: `infra/run.sh ./infra/02-cognito.sh`

Deployed resources

Type	ID / ARN	Name
AWS::Cognito::UserPool	`ap-southeast-1_kSVpAo5yl`	rsidc-users
AWS::Cognito::UserPoolClient	`kff7mg63m7d94n6f2euip41na`	rsidc-agent-client (OBO)
AWS::Cognito::UserPoolUser	`wei, zhang, chen, ds1`	test personas

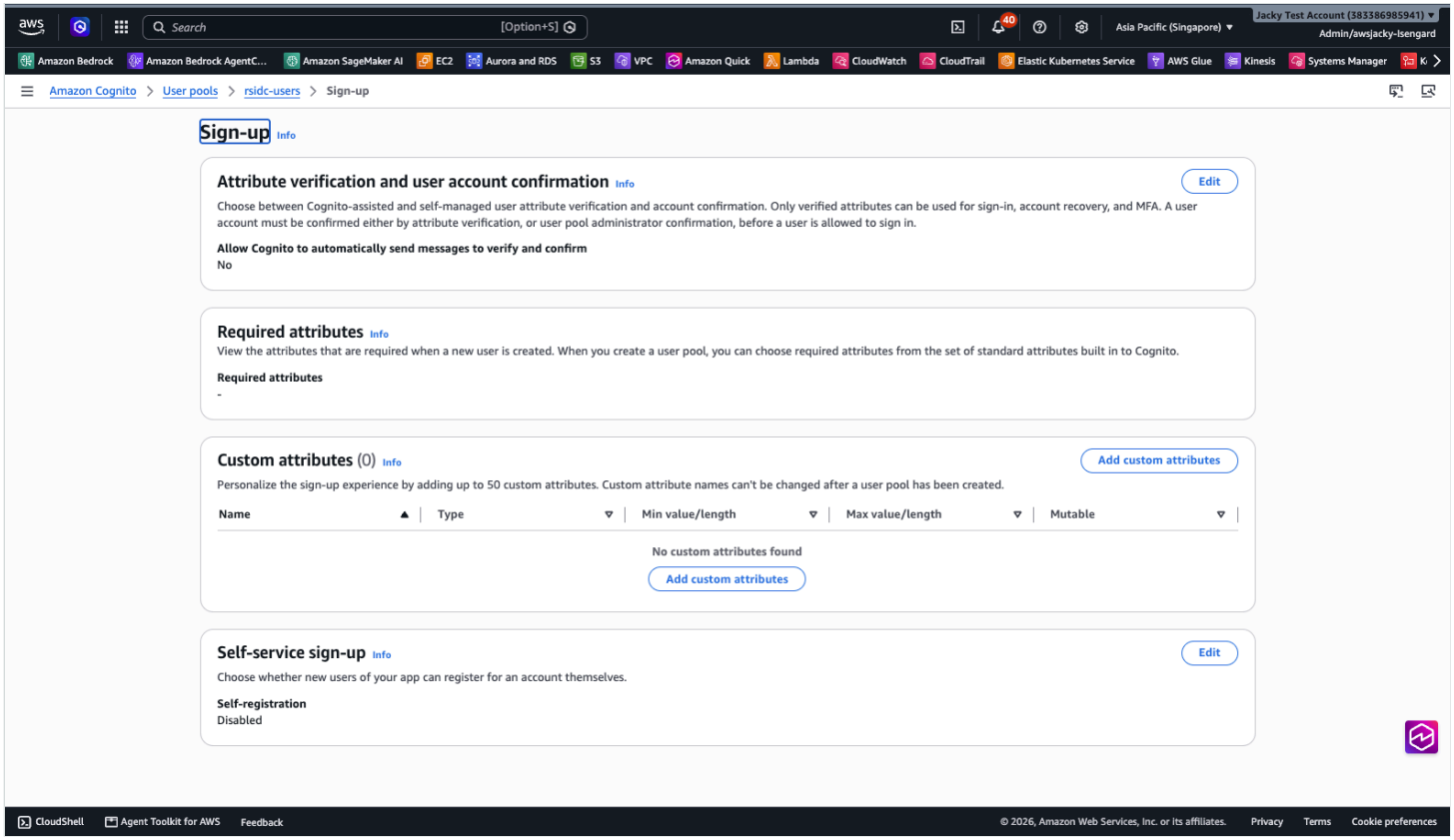
Console: Cognito pool rsidc-users: 4 admin-created users, verified emails

URL at capture (2026-10-05 16:14:31 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/user-management/users?region=ap-southeast-1



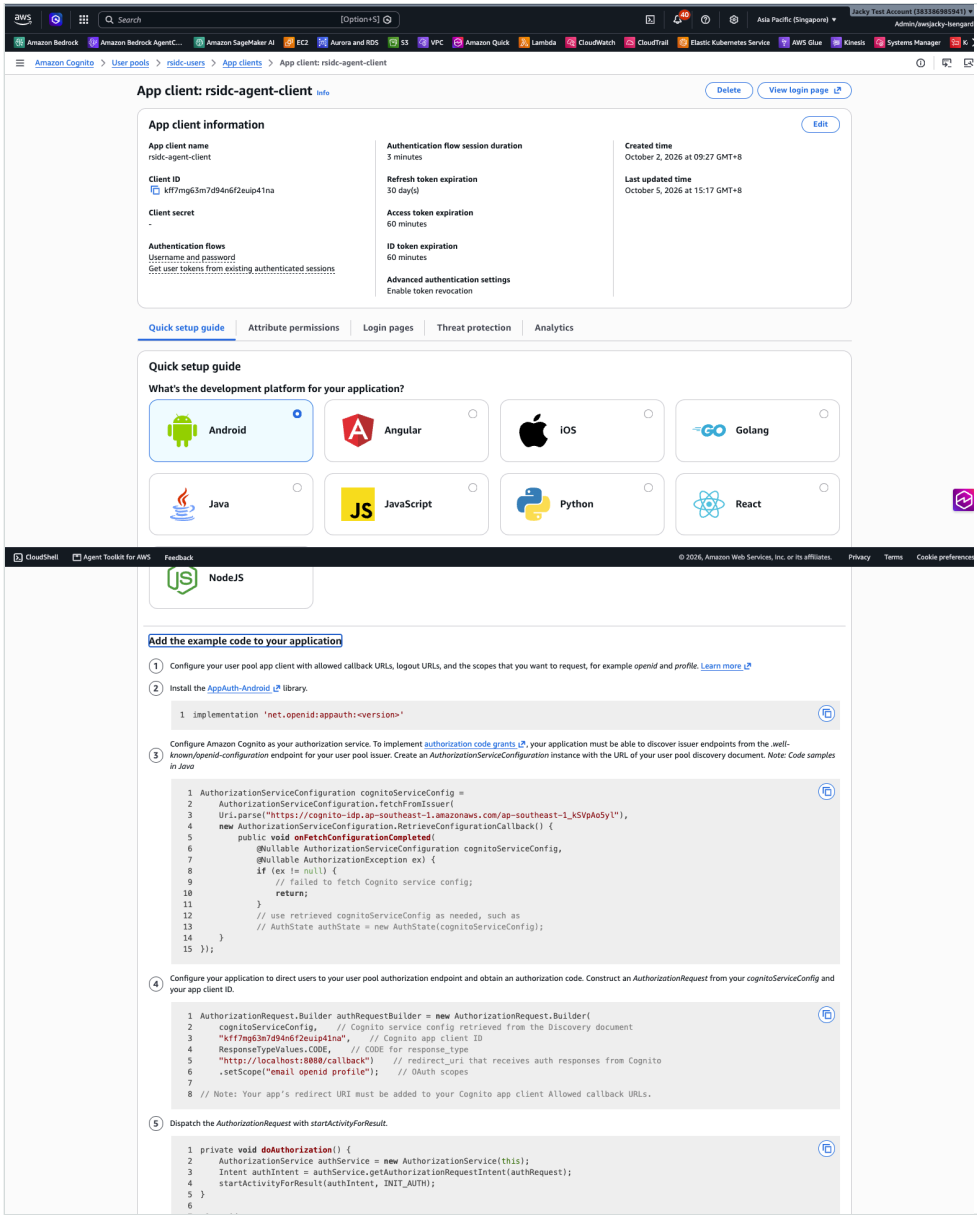
Console: Self-registration = Disabled (admin-only user creation)

URL at capture (2026-10-05 16:14:39 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/authentication/sign-up?region=ap-southeast-1



Console: OBO app client rsidc-agent-client (public, USER_PASSWORD_AUTH; its ID is the JWT audience)

URL at capture (2026-10-05 16:14:47 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/app-clients/kff7mg63m7d94n6f2euip41na/quick-setup-guide?region=ap-southeast-1



Add the example code to your application

1. Configure your user pool app client with allowed callback URLs, logout URLs, and the scopes that you want to request, for example *openid* and *profile*. Learn more

2. Install the AppAuth-Android library.

```
implementation 'net.openid:appauth:<version>'
```

3. Configure Amazon Cognito as your authorization service. To implement authorization code grants, your application must be able to discover issuer endpoints from the *.well-known/openid-configuration* endpoint for your user pool issuer. Create an *AuthorizationServiceConfiguration* instance with the URL of your user pool discovery document. *Note: Code samples in Java*

```
AuthorizationServiceConfiguration cognitoServiceConfig =
    AuthorizationServiceConfiguration.fetchFromIssuer(
    Uri.parse("https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_kSVpAo5yl"),
    new AuthorizationServiceConfiguration.RetrieveConfigurationCallback() {
        public void onFetchConfigurationCompleted(
            @Nullable AuthorizationServiceConfiguration cognitoServiceConfig,
            @Nullable AuthorizationException ex) {
            if (ex != null) {
                // failed to fetch Cognito service config;
                return;
            }
            // use retrieved cognitoServiceConfig as needed, such as
            // AuthState authState = new AuthState(cognitoServiceConfig);
    }
});
```

4. Configure your application to direct users to your user pool authorization endpoint and obtain an authorization code. Construct an *AuthorizationRequest* from your *cognitoServiceConfig* and your app client ID.

```
AuthorizationRequest.Builder authRequestBuilder = new AuthorizationRequest.Builder(
    cognitoServiceConfig,    // Cognito service config retrieved from the Discovery document
    "kff7mg63m7d94n6f2euip41na",    // Cognito app client ID
    ResponseTypeValues.CODE,    // CODE for response_type
    "http://localhost:8080/callback")    // redirect_uri that receives auth responses from Cognito
    .setScope("email openid profile");    // OAuth scopes

// Note: Your app's redirect URI must be added to your Cognito app client Allowed callback URLs.
```

5. Dispatch the *AuthorizationRequest* with *startActivityForResult*.

```
private void doAuthorization() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent authIntent = authService.getAuthorizationRequestIntent(authRequest);
    startActivityForResult(authIntent, INIT_AUTH);
}
```

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == INIT_AUTH) {
        AuthorizationResponse resp = AuthorizationResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

6 Capture the authorization redirect. After a user signs in with managed login, Amazon Cognito redirects them to the return URL. Configure a handler at the return URL for Android. Use a manifest placeholder for a custom URL scheme. If your return URL has an HTTPS URL schema, you can add an intent filter for *RedirectUriReceiverActivity* in *AndroidManifest.xml*.

```
android.defaultConfig.manifestPlaceholders = [
  'appAuthRedirectScheme': 'cognito.sample.android.demo'
]
```

7 Get the authorization response from the *fromIntent()* methods *AuthorizationResponse* or *AuthorizationException*.

```
public void onStart() {
    AuthorizationResponse response = AuthorizationResponse.fromIntent(getIntent());
    AuthorizationException ex = AuthorizationException.fromIntent(getIntent());
    if (response != null) {
        // authorization succeeded
    } else {
        // authorization failed
    }
}
```

8 Configure a handler for the *AuthorizationResponse* to exchange the authorization code for user pool tokens. Assemble a token-exchange request with *createTokenExchangeRequest* and implement *TokenResponseCallback()*. A successful exchange produces a *TokenResponse* object.

```
authService.performTokenRequest(
    authorizationResponse.createTokenExchangeRequest(),
    new AuthorizationService.TokenResponseCallback() {
        @Override
        public void onTokenRequestCompleted(TokenResponse response, AuthorizationException ex) {
            // use the response, such as authState.update(response, ex);
            if (response != null) {
                // exchange succeeded
            } else {
                // exchange failed
            }
        }
    });
```

9 Configure session-end behavior for users who sign out. Assemble an *EndSessionRequest* to your user pool logout endpoint.

```
EndSessionRequest endSessionRequest =
    new EndSessionRequest.Builder(cognitoServiceConfiguration)
        .setIdTokenHint(ID_TOKEN) // retrieve the ID_TOKEN from the TokenResponse
        .setPostLogoutRedirectUri("http://localhost:8080/")
        .setAdditionalParameters(Map.of(
             "client_id", "kff7mg63m7d94n6f2euip41na",
             "logout_uri", "http://localhost:8080/"))
        .build();
// Note: Your app's end-session redirect URI must be added to your Cognito app client Allowed sign out URLs.
```

10 Dispatch the *EndSessionRequest* with *startActivityForResult*.

```
private void endSession() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent endSessionIntent = authService.getEndSessionRequestIntent(endSessionRequest);
    startActivityForResult(endSessionIntent, END_SESSION);
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == END_SESSION) {
        EndSessionResponse resp = EndSessionResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

part 2/2

Verification

Check	Command	Expected	Result
admin-only user creation	`aws cognito-idp describe-user-pool --region $RT_REGION --user-pool-id $COGNITO_POOL_ID --query UserPool.AdminCreateUserConfig.AllowAdminCreateUserOnly`	`true`	**PASS**
4 confirmed users	`aws cognito-idp list-users --region $RT_REGION --user-pool-id $COGNITO_POOL_ID --query 'length(Users[?UserStatus==`CONFIRMED`])'`	`^4$`	**PASS**

▼ Actual output: admin-only user creation

```
true
```

▼ Actual output: 4 confirmed users

```
4
```

Step 3: IdC Trusted Token Issuer + Redshift IdC application

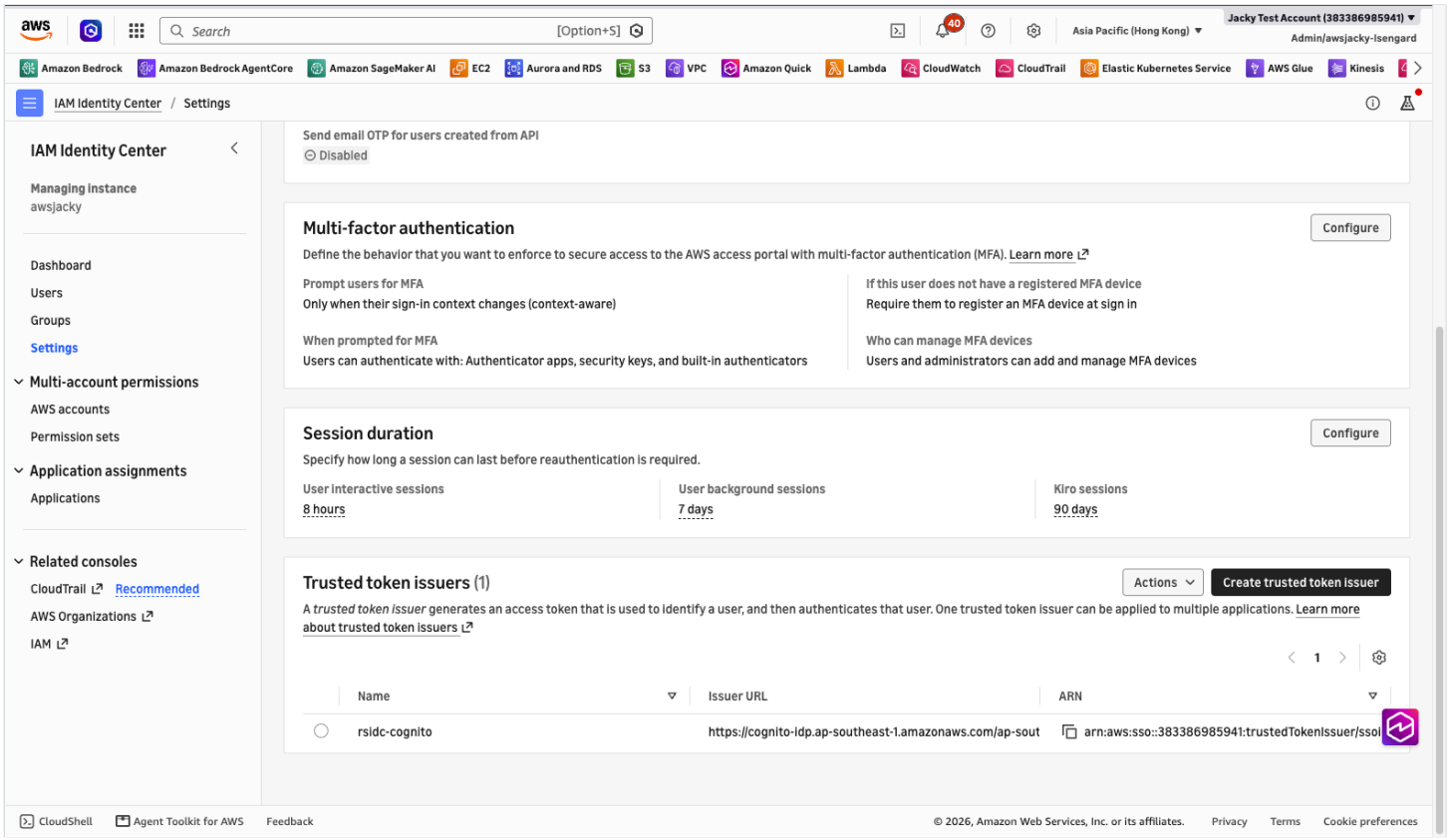
The TTI tells IAM Identity Center to trust JWTs signed by the Cognito pool and maps the `email` claim to the IdC user's `emails.value`. The Redshift IdC application (namespace AWSIDC) accepts that TTI for audience = the OBO client.

IaC / script: `infra/03-idc-redshift-app.sh`
Deploy command: `infra/run.sh ./infra/03-idc-redshift-app.sh`

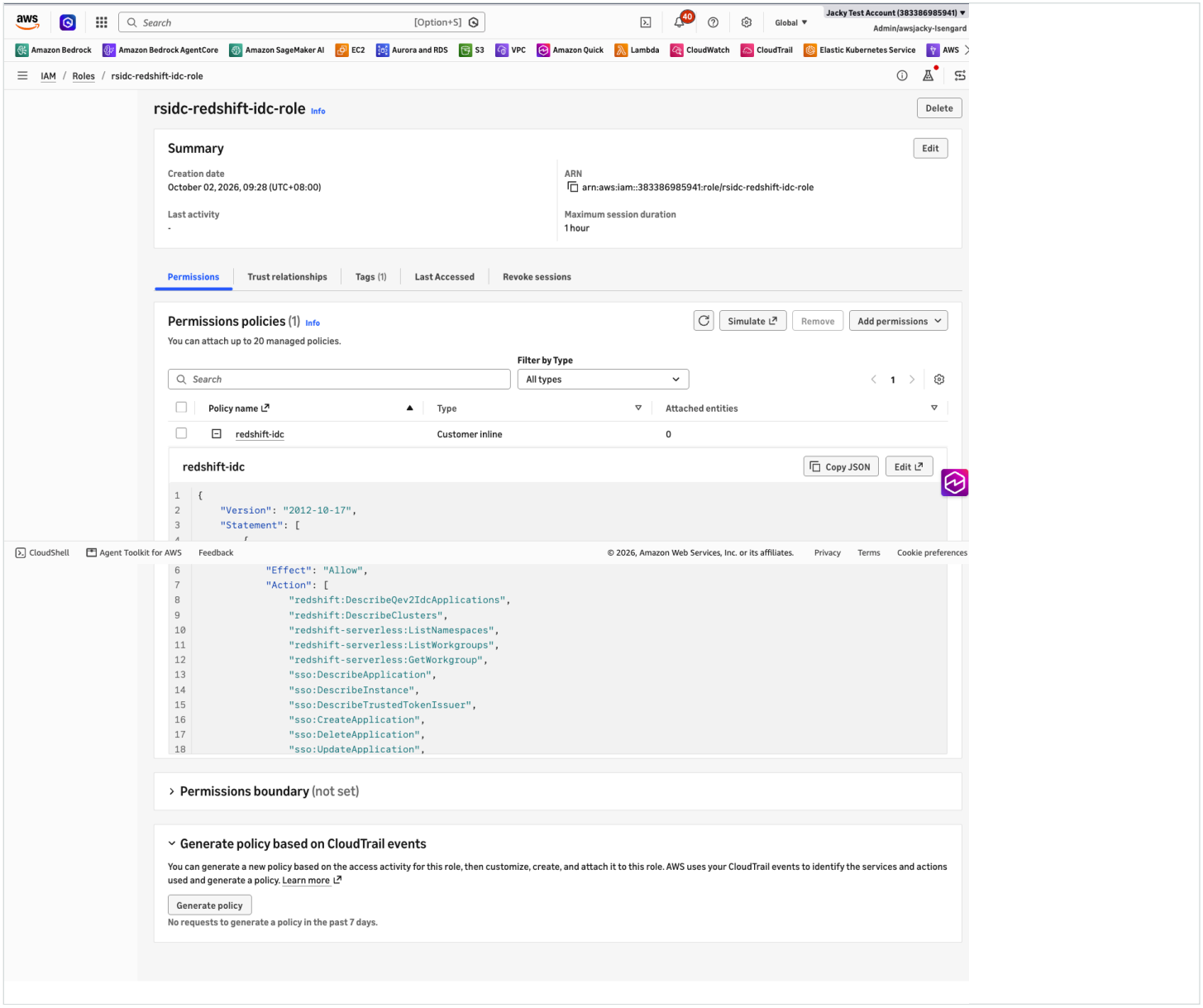
Deployed resources

Type	ID / ARN	Name
AWS::SSO::TrustedTokenIssuer	`arn:aws:sso::383386985941:trustedTokenIssuer/ssoins-715869a9a6444dc5/tti-54429cde-c0b0-70e4-eeec-ba89c6508f18`	rsidc-cognito
AWS::IAM::Role	`arn:aws:iam::383386985941:role/rsidc-redshift-idc-role`	rsidc-redshift-idc-role
AWS::Redshift::IdcApplication	`arn:aws:redshift:ap-east-1:383386985941:redshiftidcapplication:291c5e1f-f78a-45ef-9312-a430cdf65fbb`	rsidc-redshift (AWSIDC)
AWS::SSO::Application	`arn:aws:sso::383386985941:application/ssoins-715869a9a6444dc5/apl-7158c218c69d902a`	IdC-managed app for Redshift

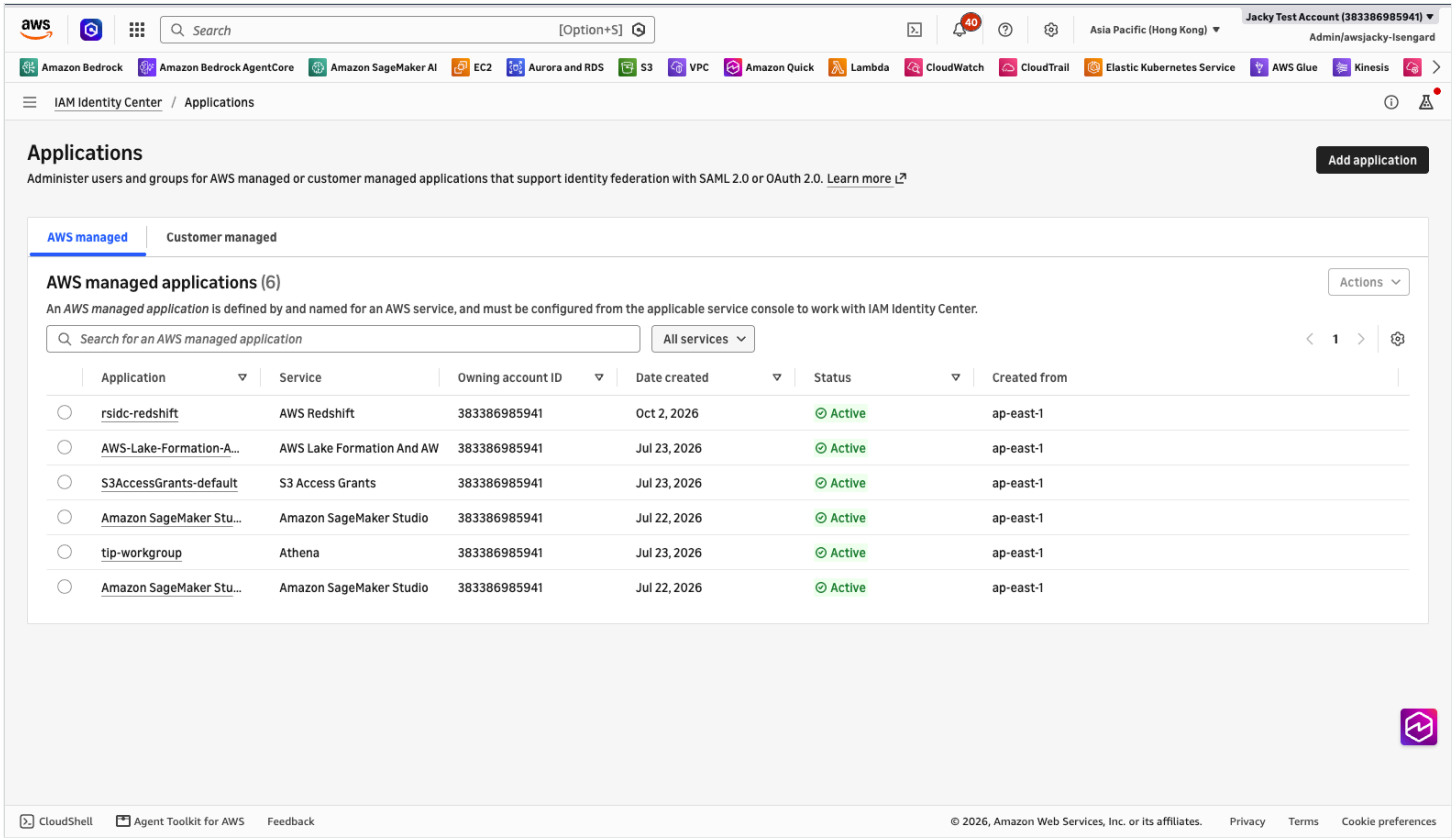
Console: IdC Settings → Authentication → Trusted token issuer rsidc-cognito (Cognito issuer, email → emails.value)
URL at capture (2026-10-05 16:15:02 HKT): https://ap-east-1.console.aws.amazon.com/singlesignon/home?region=ap-east-1&tab=network-security#/instances/715869a9a6444dc5/settings



Console: IAM role used by the Redshift IdC application
URL at capture (2026-10-05 16:15:15 HKT): https://us-east-1.console.aws.amazon.com/iam/home#/roles/details/rsidc-redshift-idc-role?section=permissions

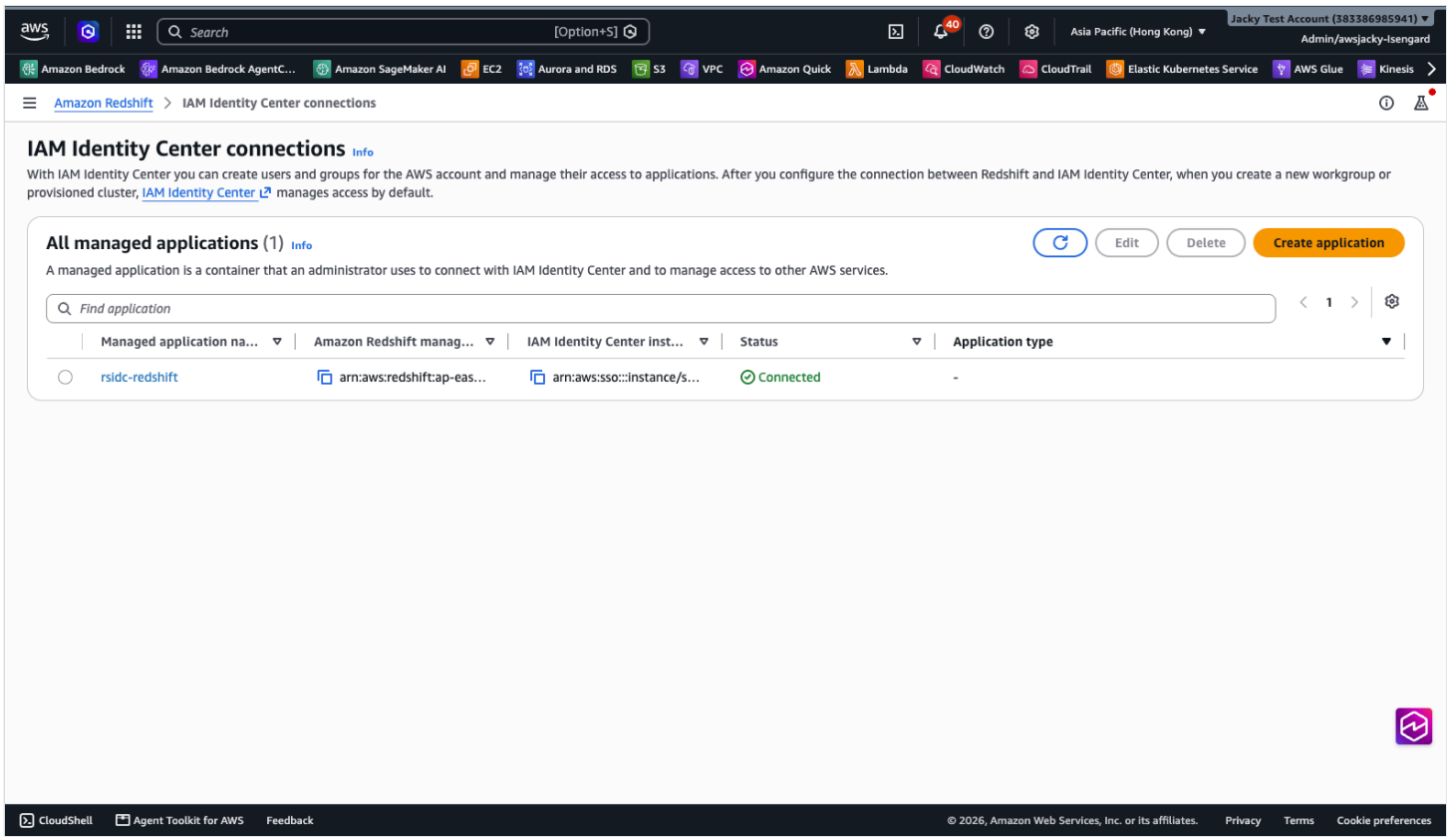


Console: IdC → Applications (AWS managed): rsidc-redshift
URL at capture (2026-10-05 16:15:23 HKT): https://ap-east-1.console.aws.amazon.com/singlesignon/home?region=ap-east-1#/instances/715869a9a6444dc5/applications



Console: Redshift → IAM Identity Center connections: rsidc-redshift (namespace AWSIDC, token issuer = Cognito TTI)

URL at capture (2026-10-05 16:25:21 HKT): https://ap-east-1.console.aws.amazon.com/redshiftv2/home?region=ap-east-1#/iam-idc-integration



Verification

Check	Command	Expected	Resul
TTI issuer + claim mapping	`aws sso-admin describe-trusted-token-issuer --region $IDC_REGION --trusted-token-issuer-arn $TTI_ARN --query TrustedTokenIssuerConfiguration.OidcJwtConfiguration`	`ap-southeast-1_kSVpAo5yl.*"ClaimAttributePath": "email".*"emails.value"`	PASS

Redshift IdC app trusts the TTI for the OBO audience	`aws redshift describe-redshift-idc-applications --region $IDC_REGION --redshift-idc-application-arn $REDSHIFT_IDC_APP_ARN --query 'RedshiftIdcApplications[0].[IdentityNamespace,IdcOnboardStatus,AuthorizedTokenIssuerList[0].AuthorizedAudiencesList[0]]' --output text`	`AWSIDC\s+Completed\s+kff7mg63m7d94n6f2euip41na`	**PASS**

▼ Actual output: TTI issuer + claim mapping

```
{
    "IssuerUrl": "https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_kSVpAo5yl",
    "ClaimAttributePath": "email",
    "IdentityStoreAttributePath": "emails.value",
    "JwksRetrievalOption": "OPEN_ID_DISCOVERY"
}
```

▼ Actual output: Redshift IdC app trusts the TTI for the OBO audience

```
AWSIDC  Completed       kff7mg63m7d94n6f2euip41na
```

Step 4: Redshift cluster, network-restricted (ap-east-1)

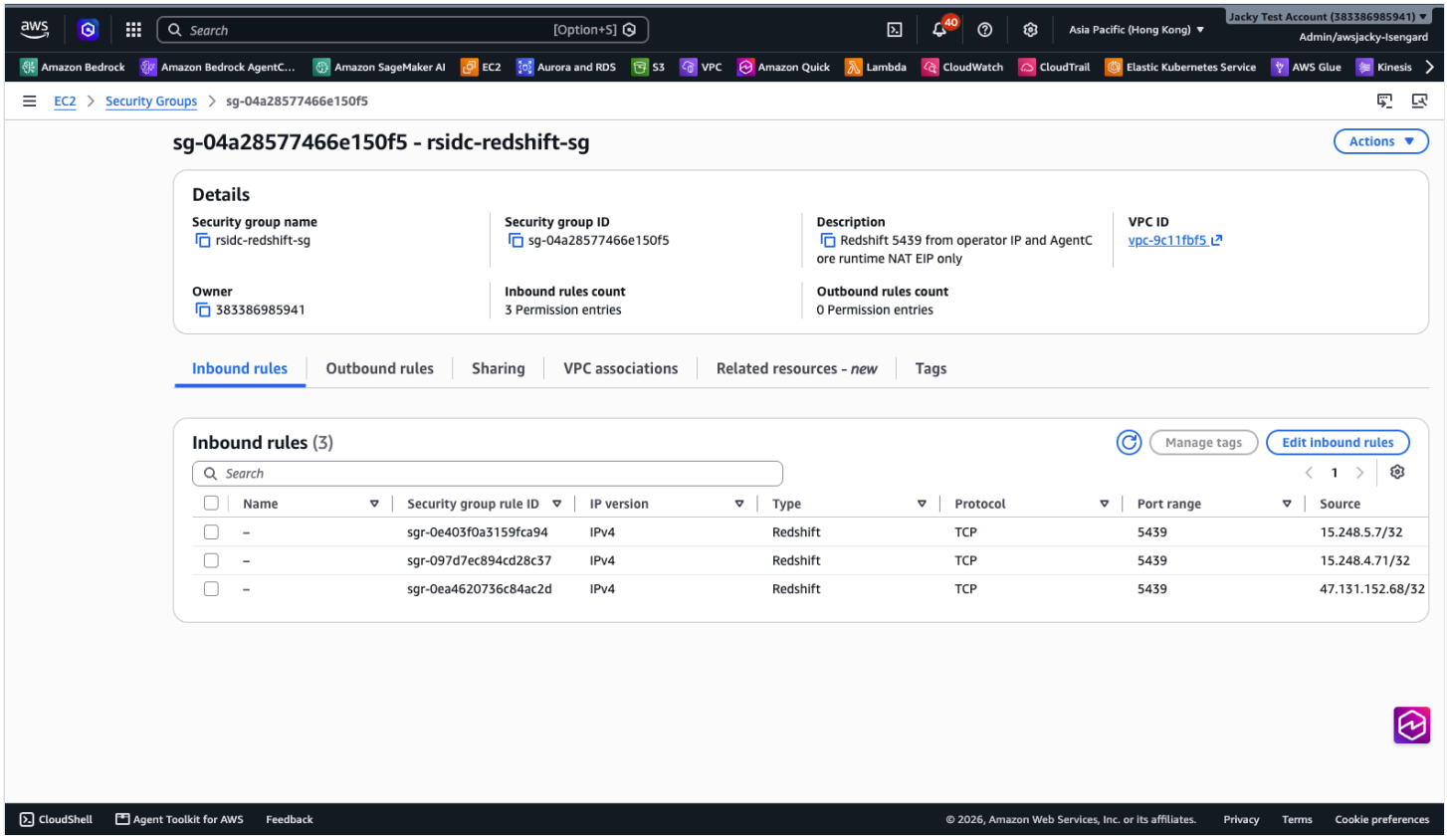
ra3.large single node attached to the IdC application. Port 5439 is open only to the operator workstation egress IPs and the runtime NAT EIP; no egress rules. The admin password is managed by Secrets Manager.

IaC / script: `infra/04-redshift-cluster.sh`
Deploy command: `infra/run.sh ./infra/04-redshift-cluster.sh`

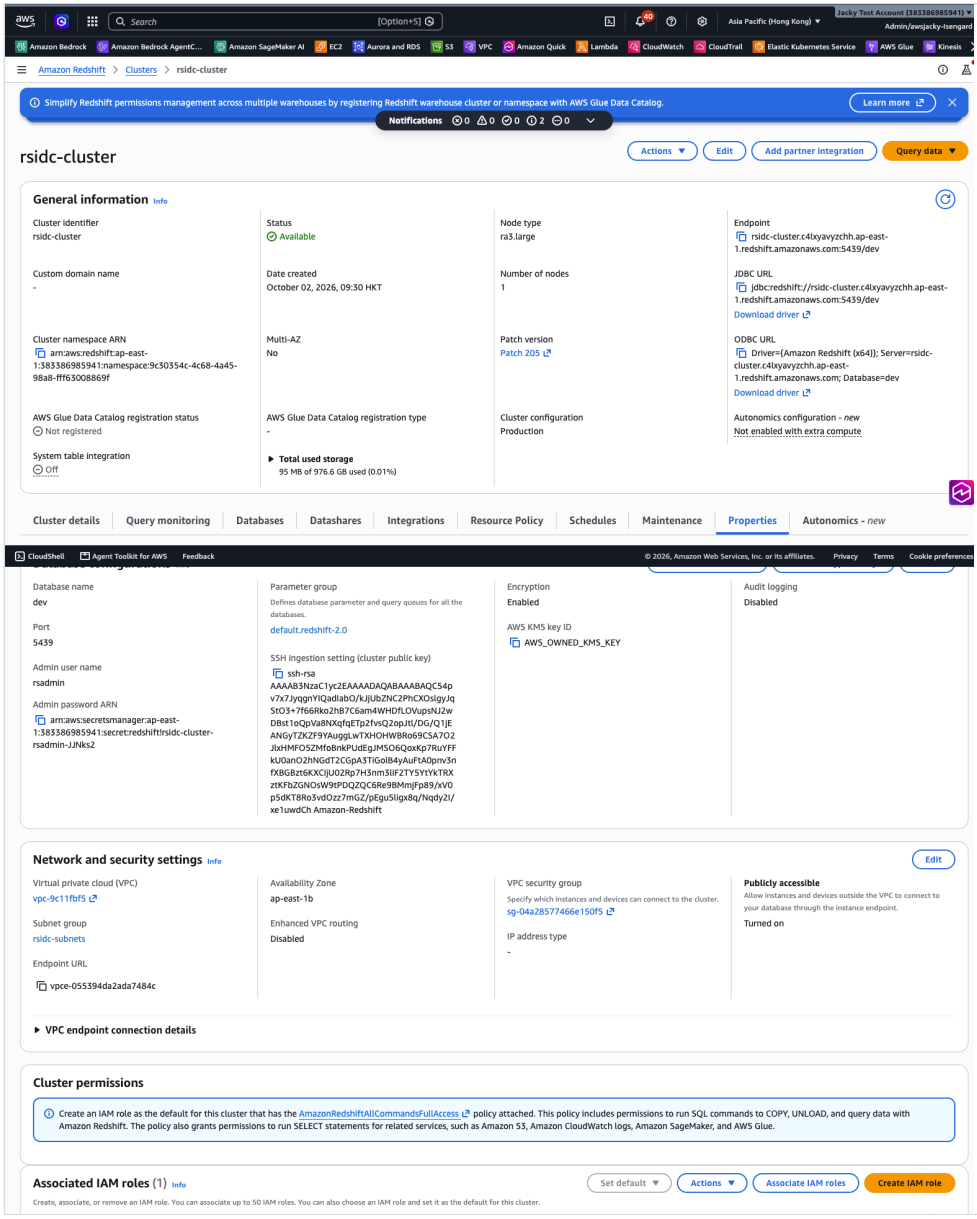
Deployed resources

Type	ID / ARN	Name
AWS::EC2::SecurityGroup	`sg-04a28577466e150f5`	rsidc-redshift-sg
AWS::Redshift::ClusterSubnetGroup	`rsidc-subnets`	default VPC subnets
AWS::Redshift::Cluster	`rsidc-cluster`	rsidc-cluster.c4lxyavyzchh.ap-east-1.redshift.amazonaws.com
AWS::SecretsManager::Secret	`arn:aws:secretsmanager:ap-east-1:383386985941:secret:redshift!rsidc-cluster-rsadmin-JJNks2`	managed admin password

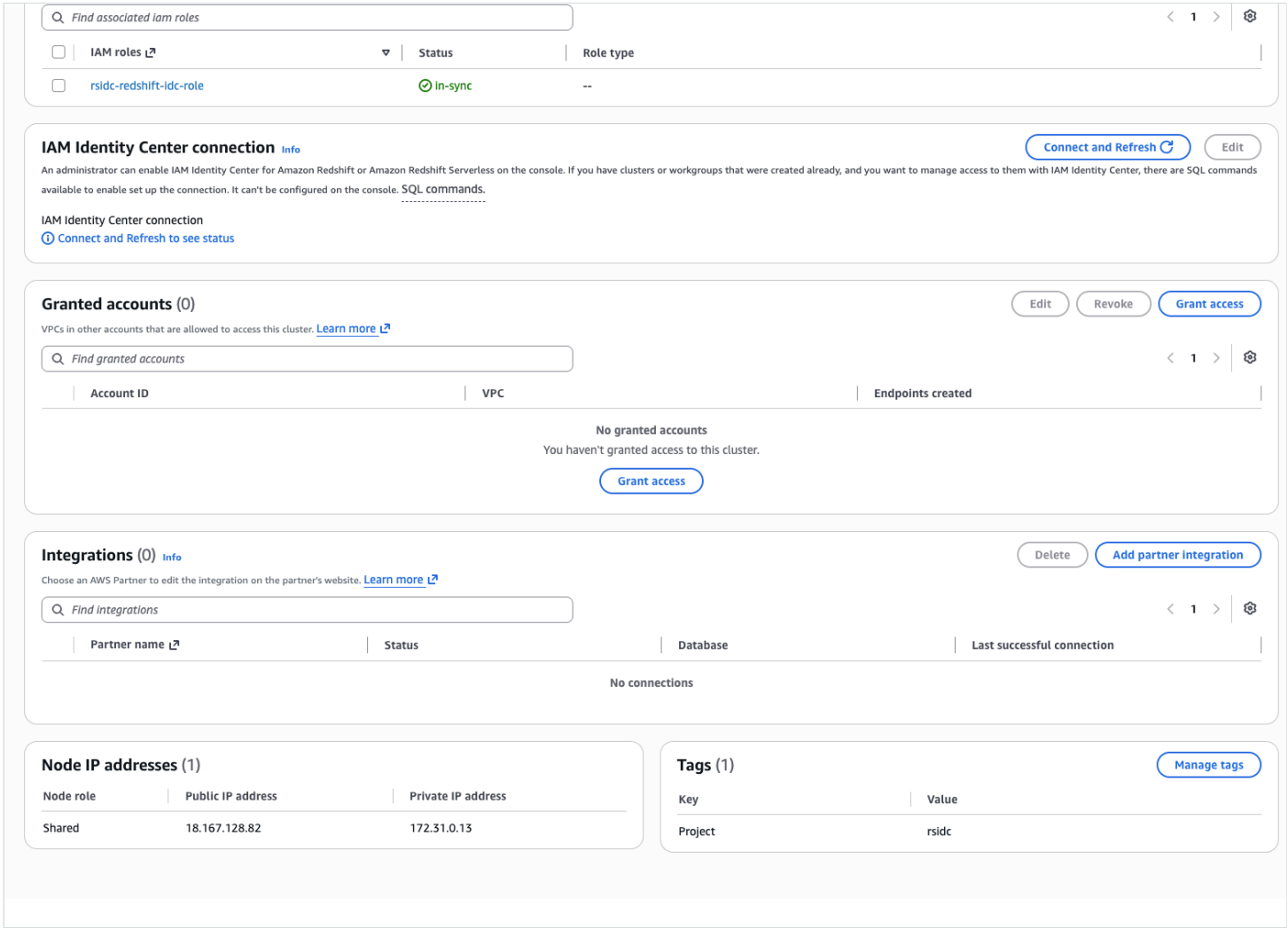
Console: Redshift security group: exactly 3 inbound /32 on 5439, 0 outbound
URL at capture (2026-10-05 16:15:41 HKT): https://ap-east-1.console.aws.amazon.com/ec2/home?region=ap-east-1#SecurityGroup:groupId=sg-04a28577466e150f5



Console: Redshift cluster rsidc-cluster (ra3.large, encrypted, IdC connection)
URL at capture (2026-10-05 16:15:50 HKT): https://ap-east-1.console.aws.amazon.com/redshiftv2/home?region=ap-east-1#/cluster-details?cluster=rsidc-cluster&tab=properties


Search
[Option+S]
Asia Pacific (Hong Kong)
Admin/awsjacky-Isengard
Amazon Redshift > Clusters > rsidc-cluster
Simplify Redshift permissions management across multiple warehouses by registering Redshift warehouse cluster or namespace with AWS Glue Data Catalog.
Learn more
Notifications
rsidc-cluster
Actions
Edit
Add partner integration
Query data
General information Info
Cluster identifier
rsidc-cluster
Status
Available
Node type
ra3.large
Endpoint
rsidc-cluster.c4lxyavyzchh.ap-east-1.redshift.amazonaws.com:5439/dev
Custom domain name
-
Date created
October 02, 2026, 09:30 HKT
Number of nodes
1
JDBC URL
jdbc:redshift://rsidc-cluster.c4lxyavyzchh.ap-east-1.redshift.amazonaws.com:5439/dev
Download driver
Cluster namespace ARN
arn:aws:redshift:ap-east-1:383386985941:namespace:9c30354c-4c68-4a45-98a8-fff63008869f
Multi-AZ
No
Patch version
Patch 205
ODBC URL
Driver={Amazon Redshift (x64)}; Server=rsidc-cluster.c4lxyavyzchh.ap-east-1.redshift.amazonaws.com; Database=dev
Download driver
AWS Glue Data Catalog registration status
Not registered
AWS Glue Data Catalog registration type
-
Cluster configuration
Production
Autonomics configuration - new
Not enabled with extra compute
System table integration
Off
Total used storage
95 MB of 976.6 GB used (0.01%)
Cluster details
Query monitoring
Databases
Datashares
Integrations
Resource Policy
Schedules
Maintenance
Properties
Autonomics - new
CloudShell
Agent Toolkit for AWS
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Database name
dev
Parameter group
Defines database parameter and query queues for all the databases.
default.redshift-2.0
Encryption
Enabled
Audit logging
Disabled
Port
5439
SSH ingestion setting (cluster public key)
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC54pv7x7JyqgnYiQadlabO/kJiUbZNC2PhCXOslgyJqStO3+7f66Rko2h87C6am4WHDfLOVupsNJ2wDBst1oQpVa8NXqfqETp2fvsQ2opJtl/DG/Q1jEANGyTZKZF9YAuggLwTXHOHWBRo69CSA7O2JlxHMFO5ZMfoBnkPUdEgJMSO6QoxKp7RuYFFkU0anO2hNGdT2CGpA3TiGolB4yAuFtAOpnv5nfXBGBzt6KXCiJU02Rp7H3nm3liF2TY5YtYkTRXztKFbZGNOsW9tPDQZQC6Re9BMmjFp89/xV0p5dKT8Ro3vdOzz7mGZ/pEgu5ligx8q/Nqdy2I/xe1uwdCh Amazon-Redshift
Admin user name
rsadmin
Admin password ARN
arn:aws:secretsmanager:ap-east-1:383386985941:secret:redshift!rsidc-cluster-rsadmin-JJNks2
Network and security settings Info
Edit
Virtual private cloud (VPC)
vpc-9c11fbf5
Availability Zone
ap-east-1b
VPC security group
Specify which instances and devices can connect to the cluster.
sg-04a28577466e150f5
Publicly accessible
Allow instances and devices outside the VPC to connect to your database through the instance endpoint.
Turned on
Subnet group
rsidc-subnets
Enhanced VPC routing
Disabled
IP address type
-
Endpoint URL
vpce-055394da2ada7484c
VPC endpoint connection details
Cluster permissions
Create an IAM role as the default for this cluster that has the AmazonRedshiftAllCommandsFullAccess policy attached. This policy includes permissions to run SQL commands to COPY, UNLOAD, and query data with Amazon Redshift. The policy also grants permissions to run SELECT statements for related services, such as Amazon S3, Amazon CloudWatch logs, Amazon SageMaker, and AWS Glue.
Associated IAM roles (1) Info
Set default
Actions
Associate IAM roles
Create IAM role
Create, associate, or remove an IAM role. You can associate up to 50 IAM roles. You can also choose an IAM role and set it as the default for this cluster.

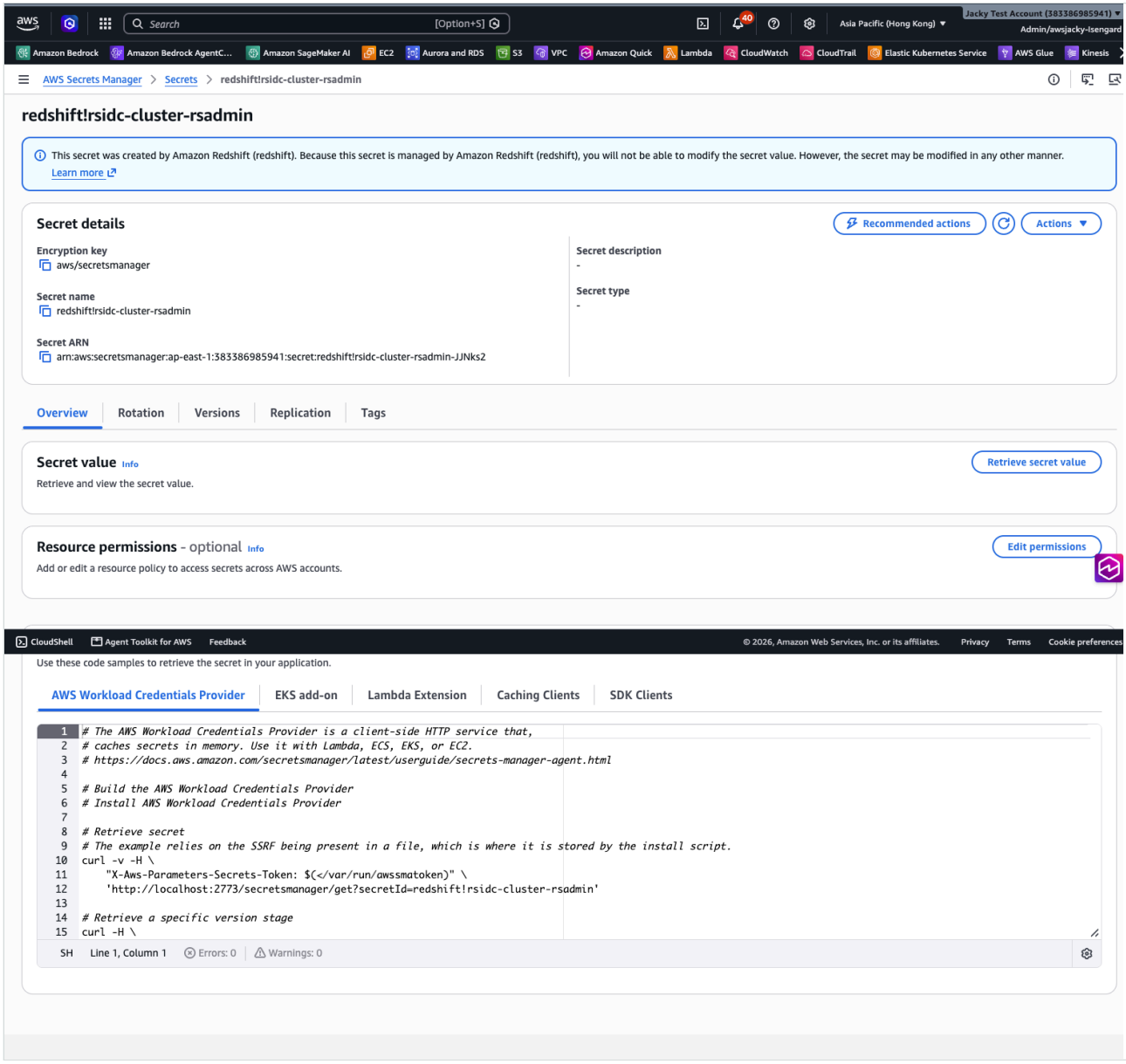
part 1/2



part 2/2

Console: Admin password managed by Secrets Manager (value not shown)

URL at capture (2026-10-05 16:16:01 HKT): https://ap-east-1.console.aws.amazon.com/secretsmanager/secret?name=redshift%21rsidc-cluster-rsadmin&region=ap-east-1#



Verification

Check	Command	Expected	Result
SG ingress = 3 × /32 on 5439, no 0.0.0.0/0	`aws ec2 describe-security-groups --region $IDC_REGION --group-ids $RS_SG_ID --query 'SecurityGroups[0].IpPermissions[].IpRanges[].CidrIp' --output text | tr '\t' '\n' | sort | tr '\n' ' '`	`^15\.248\.4\.71/32 15\.248\.5\.7/32 47\.131\.152\.68/32\s*$`	PASS
cluster available, encrypted, IdC-attached	`aws redshift describe-clusters --region $IDC_REGION --cluster-identifier rsidc-cluster --query 'Clusters[0].[ClusterStatus,NodeType,Encrypted,length(IamRoles)]' --output text`	`available\s+ra3.large\s+True\s+1`	PASS

▼ Actual output: SG ingress = 3 × /32 on 5439, no 0.0.0.0/0

```
15.248.4.71/32 15.248.5.7/32 47.131.152.68/32
```

▼ Actual output: cluster available, encrypted, IdC-attached

```
available       ra3.large       True    1
```

Step 5: Seed data + grants to IdC group roles (SQL)

Tables in `dev` (sales.orders, marketing.campaigns, risk.credit_limits), roles `AWSIDC:<group>`, table/column grants and RLS. No console page; verified with the Redshift Data API (read-only SELECT).

IaC / script: `infra/sql/01-seed-and-grants.sql`
Deploy command: `infra/run.sh agent/.venv/bin/python infra/05-seed-redshift.py`

Deployed resources

Type	ID / ARN	Name
Redshift schema/table	`dev.sales.orders / dev.marketing.campaigns / dev.risk.credit_limits`	5 + 2 + 2 rows
Redshift role	`AWSIDC:risk-team, marketing-team, platform-team, data-scientists`	grants + RLS

Verification

Check	Command	Expected	Result
IdC group roles exist	`agent/.venv/bin/python -c "import os,time,boto3;c=boto3.Session(profile_name='default').client('redshift-data',region_name=os.environ['IDC_REGION']);i=c.execute_statement(ClusterIdentifier='rsidc-cluster',Database='dev',SecretArn=os.environ['REDSHIFT_ADMIN_SECRET_ARN'],Sql=\"SELECT role_name FROM svv_roles WHERE role_name LIKE 'AWSIDC:%' ORDER BY 1\")['Id'] while c.describe_statement(Id=i)['Status'] not in ('FINISHED','FAILED'): time.sleep(.5) print(' '.join(r[0]['stringValue'] for r in c.get_statement_result(Id=i)['Records']))"`	`AWSIDC:data-scientists AWSIDC:marketing-team AWSIDC:platform-team AWSIDC:risk-team`	**PASS**
RLS enabled on sales.orders	`agent/.venv/bin/python -c "import os,time,boto3;c=boto3.Session(profile_name='default').client('redshift-data',region_name=os.environ['IDC_REGION']);i=c.execute_statement(ClusterIdentifier='rsidc-cluster',Database='dev',SecretArn=os.environ['REDSHIFT_ADMIN_SECRET_ARN'],Sql=\"SELECT relschema, relname, is_rls_on FROM svv_rls_relation WHERE relname='orders'\")['Id'] while c.describe_statement(Id=i)['Status'] not in ('FINISHED','FAILED'): time.sleep(.5) print(c.get_statement_result(Id=i)['Records'])"`	`'sales'.*'orders'.*'stringValue': 't'`	**PASS**

▼ Actual output: IdC group roles exist

```
AWSIDC:data-scientists AWSIDC:marketing-team AWSIDC:platform-team AWSIDC:risk-team
```

▼ Actual output: RLS enabled on sales.orders

```
[[{'stringValue': 'sales'}, {'stringValue': 'orders'}, {'stringValue': 't'}]]
```

Step 6: OBO runtime execution role (no Redshift permissions)

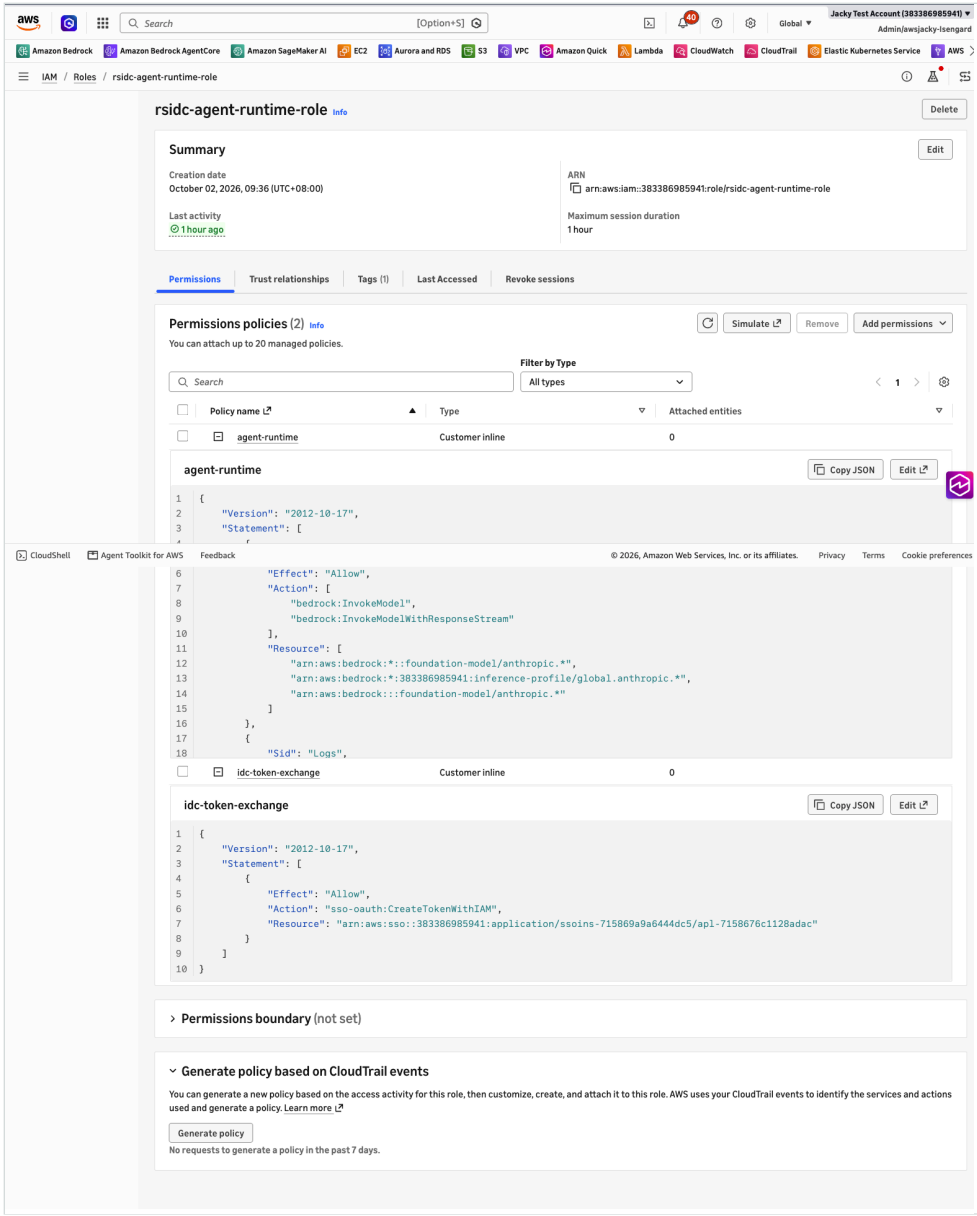
Role assumed by the OBO runtime. It can call Bedrock, write logs, use its workload identity, and (added in step 7) exchange tokens at one IdC application. It has no `redshift:*` permission.

IaC / script: `infra/06-runtime-role.sh + infra/policies/agent-runtime-permissions.json`
Deploy command: `infra/run.sh ./infra/06-runtime-role.sh`

Deployed resources

Type	ID / ARN	Name
AWS::IAM::Role	`arn:aws:iam::383386985941:role/rsidc-agent-runtime-role`	rsidc-agent-runtime-role

Console: OBO runtime role: Bedrock, logs, workload identity, CreateTokenWithIAM on one IdC app; no redshift:*
URL at capture (2026-10-05 16:16:14 HKT): https://us-east-1.console.aws.amazon.com/iam/home#/roles/details/rsidc-agent-runtime-role?section=permissions



part 1/2

part 2/2

Verification

Check	Command	Expected	Result
no redshift actions in any inline policy	`for p in $(aws iam list-role-policies --role-name rsidc-agent-runtime-role --query PolicyNames --output text); do aws iam get-role-policy --role-name rsidc-agent-runtime-role --policy-name $p --query PolicyDocument; done | grep -c 'redshift:' ; true`	`^0$`	PASS

▼ Actual output: no redshift actions in any inline policy

```
0
```

Step 7: IdC customer-managed app for the agent's token exchange

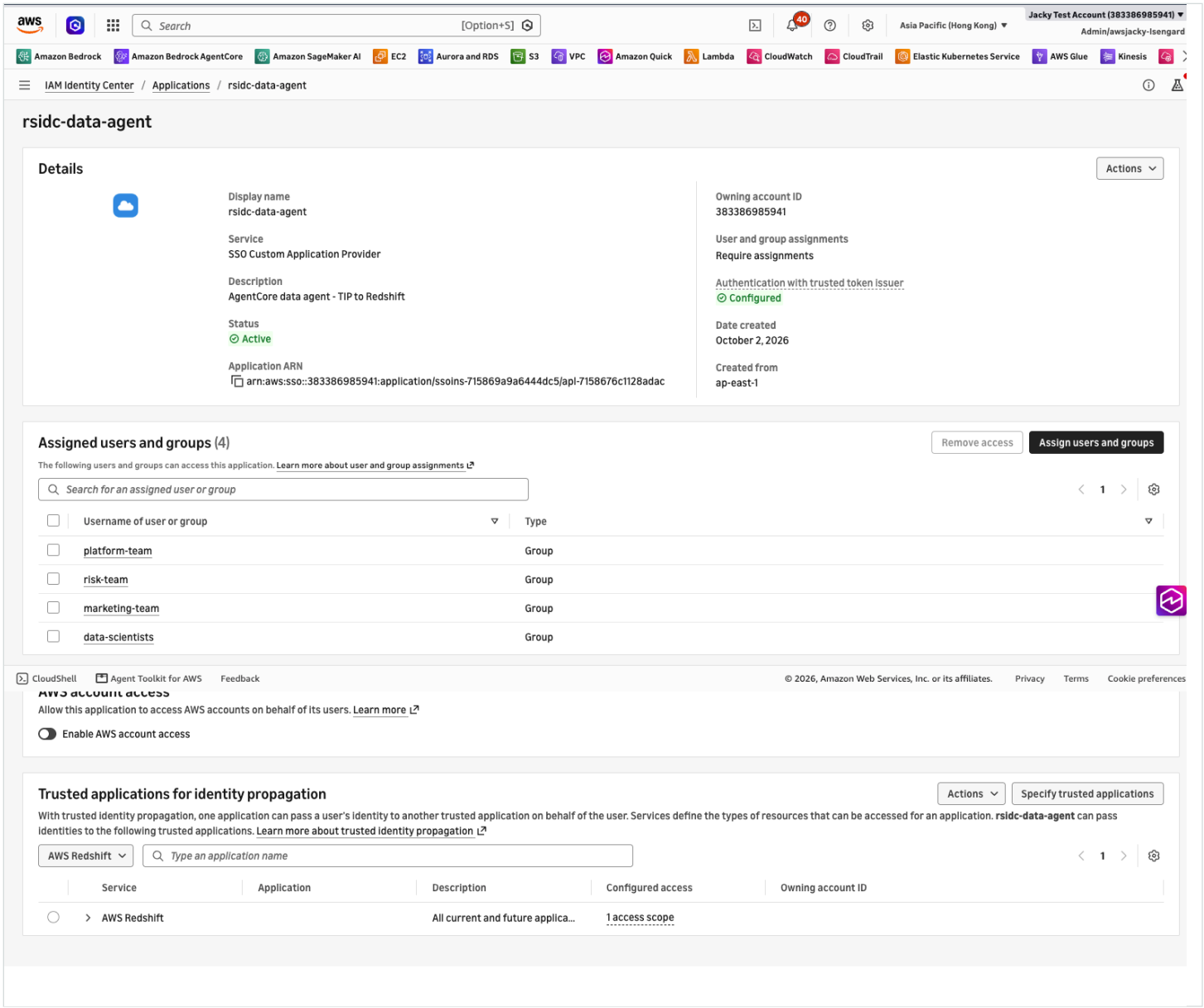
`rsidc-data-agent` has a jwt-bearer grant trusting the Cognito TTI (audience = OBO client), scope `redshift:connect`, required group assignment, and an ActorPolicy that allows only the OBO runtime role and explicitly denies every other principal.

IaC / script: `infra/07-idc-agent-app.sh`
Deploy command: `infra/run.sh ./infra/07-idc-agent-app.sh`

Deployed resources

Type	ID / ARN	Name
AWS::SSO::Application	`arn:aws:sso::383386985941:application/ssoins-715869a9a6444dc5/apl-7158676c1128adac`	rsidc-data-agent

Console: IdC customer-managed app rsidc-data-agent: TTI configured, 4 groups, trusted for AWS Redshift
URL at capture (2026-10-05 16:16:34 HKT): https://ap-east-1.console.aws.amazon.com/singlesignon/home?region=ap-east-1#/instances/715869a9a6444dc5/applications/ins-7158676c1128adac



Verification

Check	Command	Expected	Result
jwt-bearer grant → Cognito TTI, OBO audience	`aws sso-admin get-application-grant --region $IDC_REGION --application-arn $AGENT_IDC_APP_ARN --grant-type urn:ietf:params:oauth:grant-type:jwt-bearer --query 'Grant.JwtBearer.AuthorizedTokenIssuers[0].AuthorizedAudiences[0]' --output text`	`kff7mg63m7d94n6f2euip41na`	PASS
ActorPolicy: Allow runtime role + Deny everyone else	`aws sso-admin get-application-authentication-method --region $IDC_REGION --application-arn $AGENT_IDC_APP_ARN --authentication-method-type IAM --query 'AuthenticationMethod.Iam.ActorPolicy.Statement[].[Effect,Sid]' --output text`	`Allow.*Deny\s+DenyEveryoneElseIncludingAdmins`	PASS

scope redshift:connect	`aws sso-admin list-application-access-scopes --region $IDC_REGION --application-arn $AGENT_IDC_APP_ARN --query 'Scopes[].Scope' --output text`	`redshift:connect`	**PASS**

▼ Actual output: jwt-bearer grant → Cognito TTI, OBO audience

```
kff7mg63m7d94n6f2euip41na
```

▼ Actual output: ActorPolicy: Allow runtime role + Deny everyone else

```
Allow   None
Deny    DenyEveryoneElseIncludingAdmins
```

▼ Actual output: scope redshift:connect

```
redshift:connect
```

Step 8: OBO data agent on AgentCore Runtime

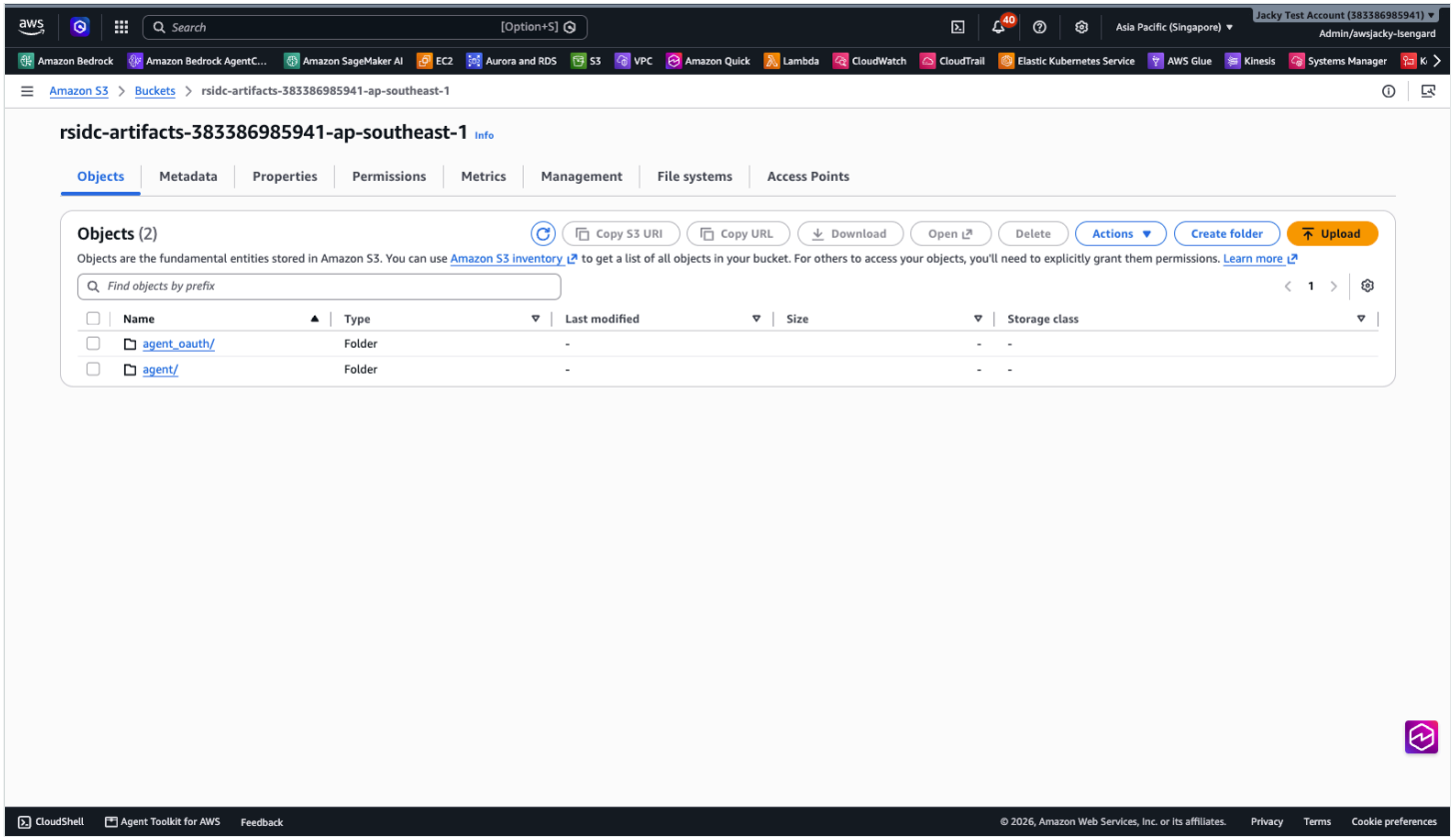
Direct code deploy (zip in S3). VPC mode in the private subnets, JWT inbound auth (Cognito discovery, audience = OBO client), Authorization header forwarded to the agent.

IaC / script: `infra/08-deploy-runtime.sh + agent/`
Deploy command: `infra/run.sh ./infra/08-deploy-runtime.sh`

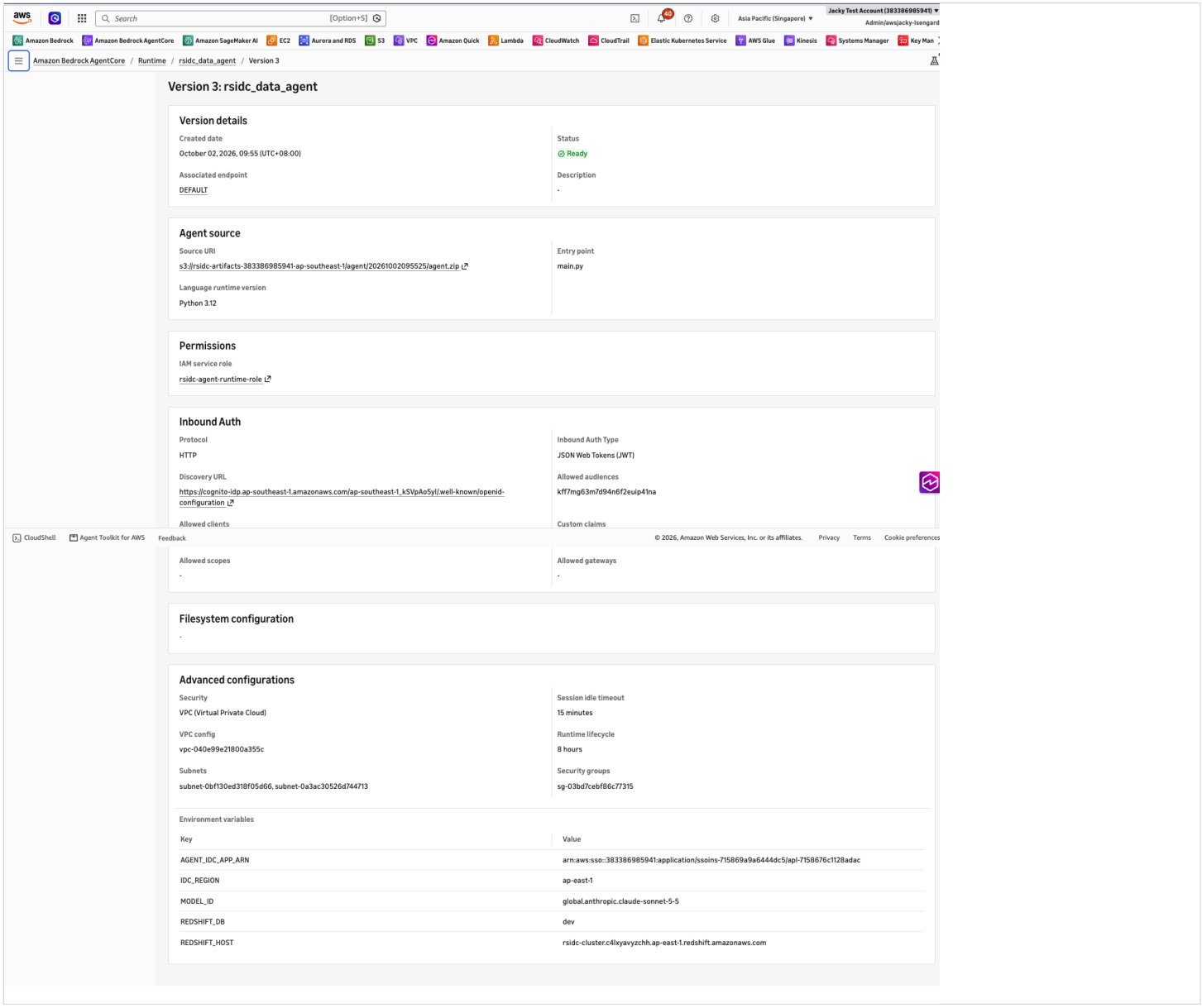
Deployed resources

Type	ID / ARN	Name
AWS::S3::Bucket	`rsidc-artifacts-383386985941-ap-southeast-1`	code bundles
AWS::BedrockAgentCore::Runtime	`arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:runtime/rsidc_data_agent-BoemCIApSW`	rsidc_data_agent (v3)

Console: S3 bucket with the runtime code bundles (agent/, agent_oauth/)
URL at capture (2026-10-05 16:16:44 HKT): https://ap-southeast-1.console.aws.amazon.com/s3/buckets/rsidc-artifacts-383386985941-ap-southeast-1?tab=objects&region=ap-southeast-1



Console: OBO runtime rsidc_data_agent v3: JWT inbound (aud kff7mg63…), VPC, env vars
URL at capture (2026-10-05 16:16:52 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/agents/rsidc_data_agent-BoemCIApSW/version/3?region=ap-southeast-1



Verification

Check	Command
runtime READY, VPC, JWT audience, header allowlist	`aws bedrock-agentcore-control get-agent-runtime --region $RT_REGION --agent-runtime-id $RT_ID --query '[status,agentRuntimeVersion,networkConfiguration.networkMode,authorizerConfiguration.customJWTAuthorizer.allowedAudience[0],requestHeaderConfigurat --output text`
bucket blocks public access	`aws s3api get-public-access-block --bucket rsidc-artifacts-383386985941-ap-southeast-1 --query 'PublicAccessBlockConfiguration.[BlockPublicAcls,BlockPublicPolicy,IgnorePublicAcls,RestrictPublicBuckets]' --output text`

▼ Actual output: runtime READY, VPC, JWT audience, header allowlist

```
READY   3       VPC     kff7mg63m7d94n6f2euip41na       Authorization
```

▼ Actual output: bucket blocks public access

```
True    True    True    True
```

Step 9: Cognito OAuth clients for 2LO / 3LO (same pool)

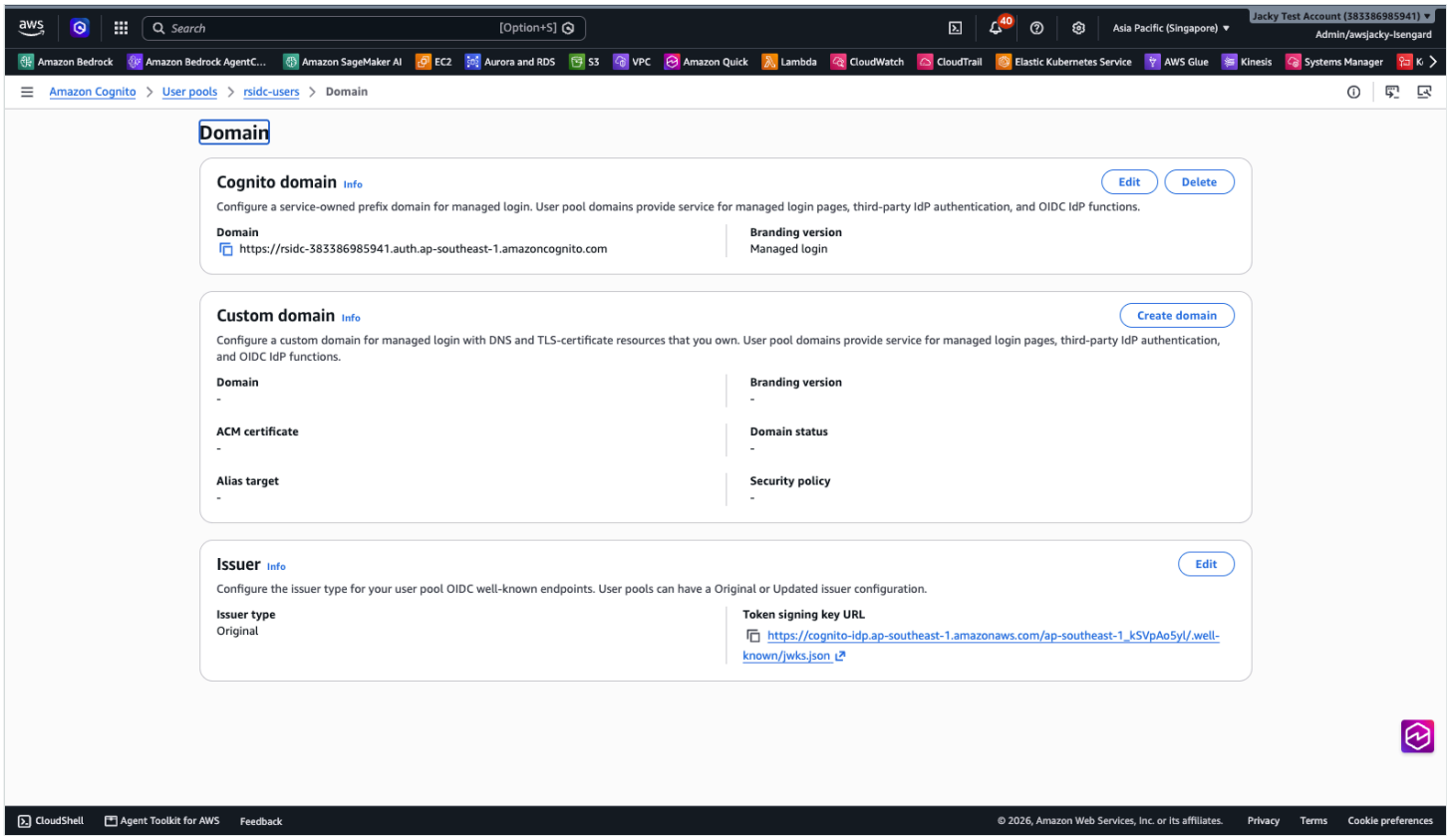
Managed login domain (v2), resource server `rsidc-reports/read`, a confidential client_credentials client (2LO) and a confidential authorization-code client (3LO). The OBO client is unchanged here.

IaC / script: `infra/09-cognito-oauth-clients.sh`
Deploy command: `infra/run.sh ./infra/09-cognito-oauth-clients.sh`

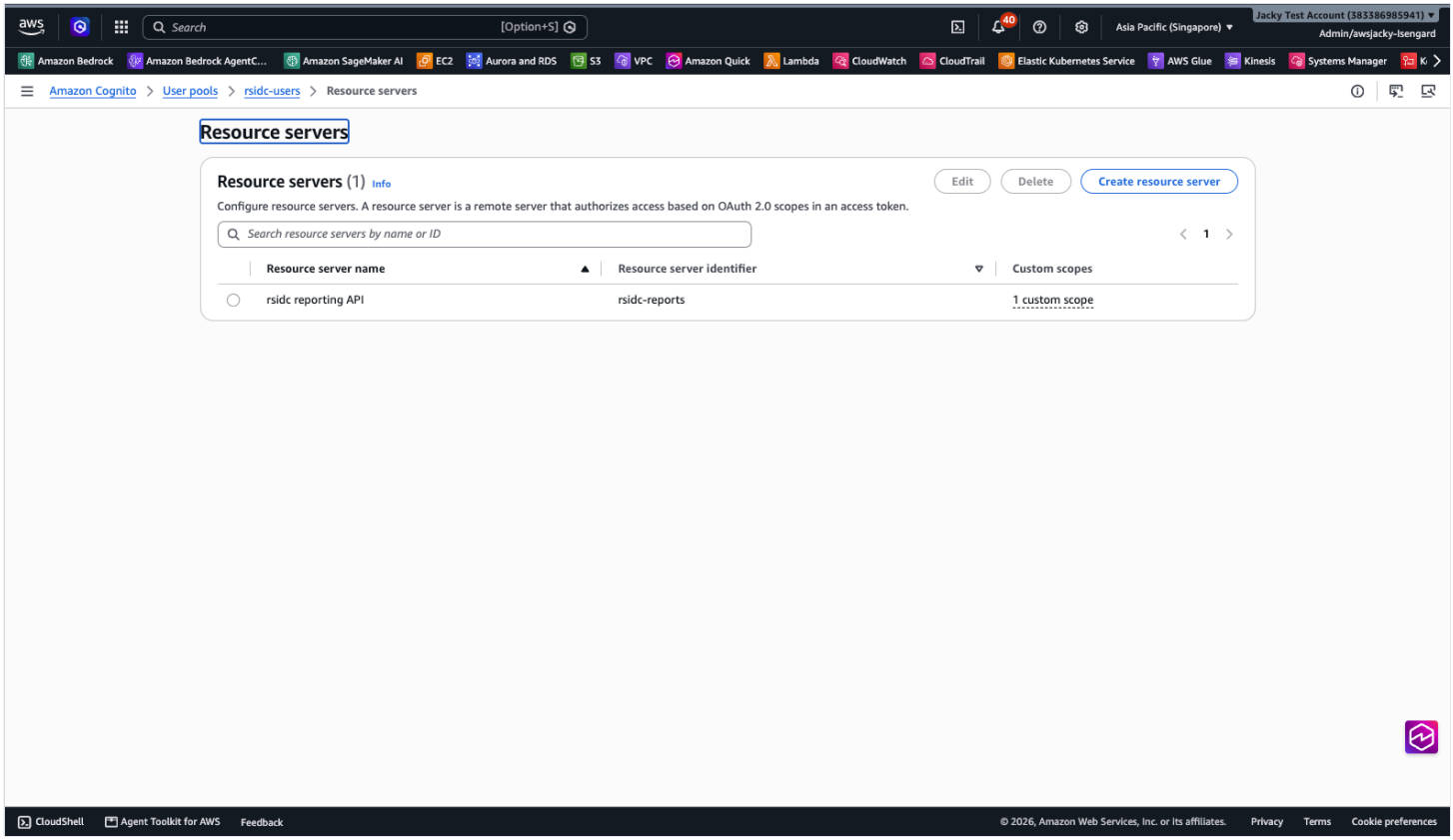
Deployed resources

Type	ID / ARN	Name
AWS::Cognito::UserPoolDomain	`rsidc-383386985941`	managed login v2
AWS::Cognito::UserPoolResourceServer	`rsidc-reports`	scope read
AWS::Cognito::UserPoolClient	`3lsr6jpphlql4hb8n1ge6a8sjp`	rsidc-2lo-agent
AWS::Cognito::UserPoolClient	`4mrecmrv4divtcngq62to3run1`	rsidc-3lo-agent

Console: Cognito managed login domain rsidc-383386985941 (Managed login v2)
URL at capture (2026-10-05 16:17:00 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/branding/domain?region=ap-southeast-1

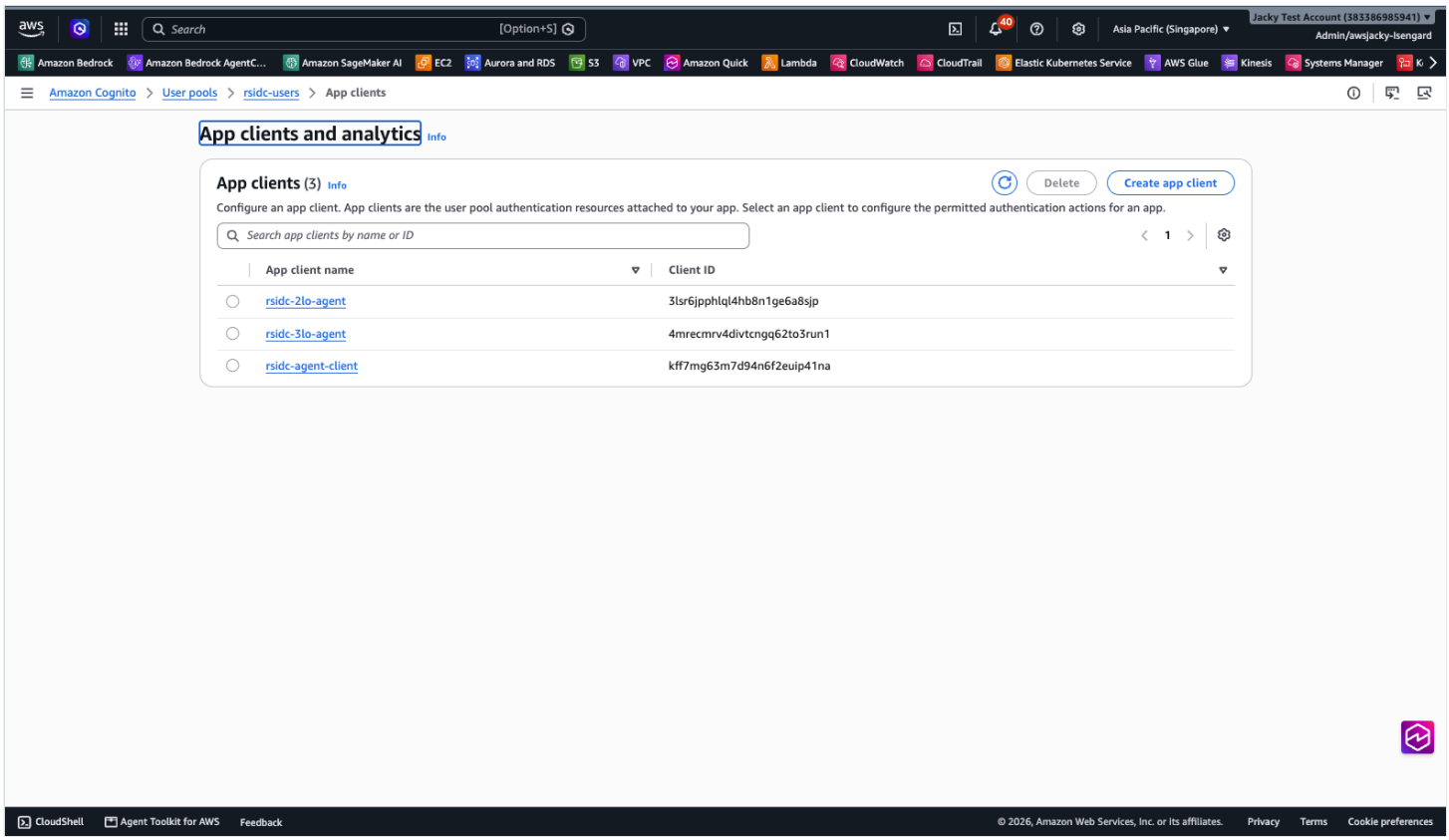


Console: Resource server rsidc-reports (scope read) for 2LO
URL at capture (2026-10-05 16:17:08 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/resource-servers?region=ap-southeast-1



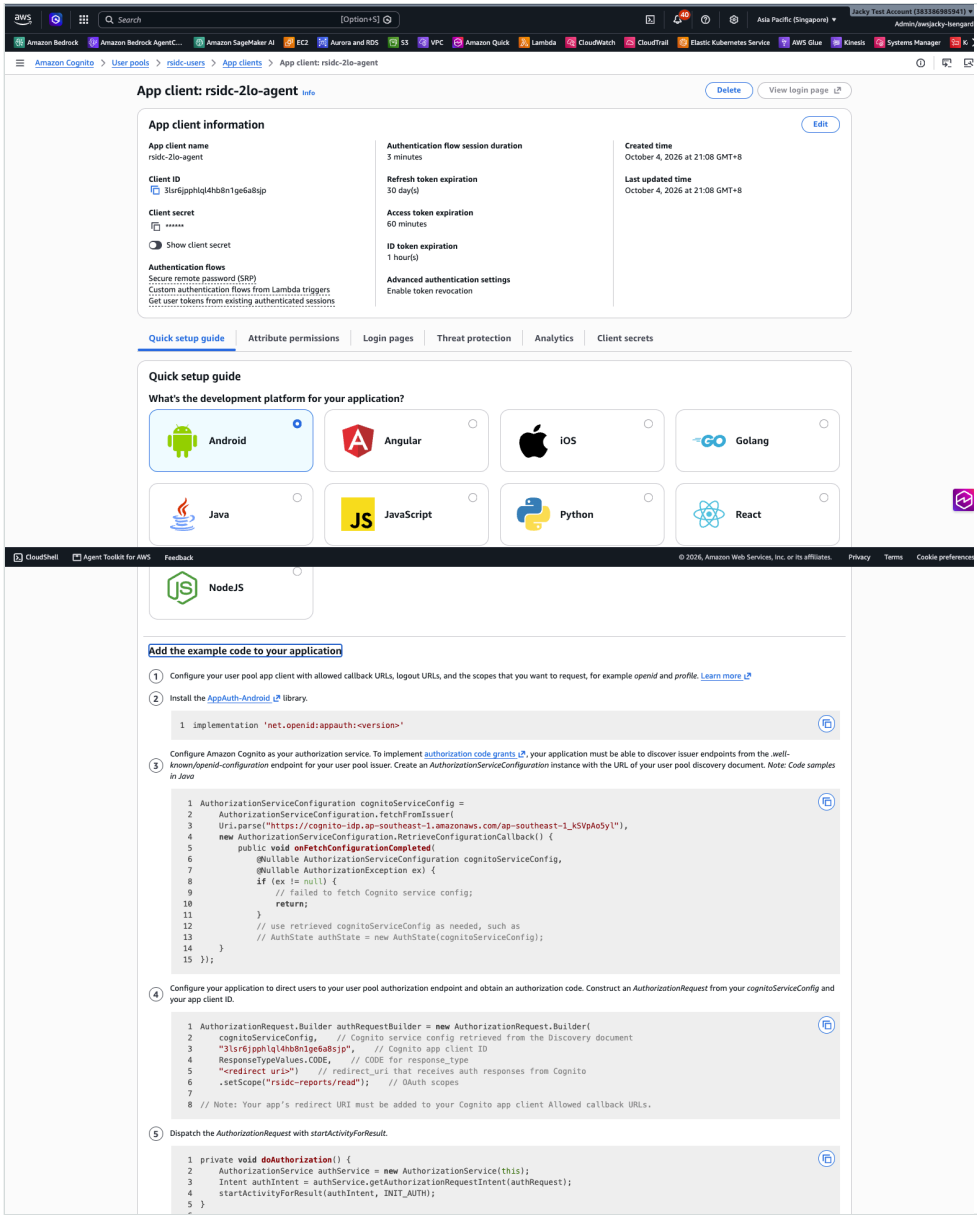
Console: App clients: OBO client, rsidc-2lo-agent, rsidc-3lo-agent

URL at capture (2026-10-05 16:17:16 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/app-clients?region=ap-southeast-1



Console: 2LO client: client_credentials, scope rsidc-reports/read

URL at capture (2026-10-05 16:17:24 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/app-clients/3lsr6jpphlql4hb8n1ge6a8sjp/quick-setup-guide?region=ap-southeast-1



Add the example code to your application

1. Configure your user pool app client with allowed callback URLs, logout URLs, and the scopes that you want to request, for example *openid* and *profile*. Learn more

2. Install the AppAuth-Android library.

```
implementation 'net.openid:appauth:<version>'
```

3. Configure Amazon Cognito as your authorization service. To implement authorization code grants, your application must be able to discover issuer endpoints from the *.well-known/openid-configuration* endpoint for your user pool issuer. Create an *AuthorizationServiceConfiguration* instance with the URL of your user pool discovery document. *Note: Code samples in Java*

```
AuthorizationServiceConfiguration cognitoServiceConfig =
    AuthorizationServiceConfiguration.fetchFromIssuer(
    Uri.parse("https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_kSVpAo5yl"),
    new AuthorizationServiceConfiguration.RetrieveConfigurationCallback() {
        public void onFetchConfigurationCompleted(
            @Nullable AuthorizationServiceConfiguration cognitoServiceConfig,
            @Nullable AuthorizationException ex) {
            if (ex != null) {
                // failed to fetch Cognito service config;
                return;
            }
            // use retrieved cognitoServiceConfig as needed, such as
            // AuthState authState = new AuthState(cognitoServiceConfig);
    }
});
```

4. Configure your application to direct users to your user pool authorization endpoint and obtain an authorization code. Construct an *AuthorizationRequest* from your *cognitoServiceConfig* and your app client ID.

```
AuthorizationRequest.Builder authRequestBuilder = new AuthorizationRequest.Builder(
    cognitoServiceConfig,     // Cognito service config retrieved from the Discovery document
    "3lsr6jpphlql4hb8n1ge6a8sjp",     // Cognito app client ID
    ResponseTypeValues.CODE,    // CODE for response_type
    "<redirect uri>")    // redirect_uri that receives auth responses from Cognito
    .setScope("rsidc-reports/read");    // OAuth scopes

// Note: Your app's redirect URI must be added to your Cognito app client Allowed callback URLs.
```

5. Dispatch the *AuthorizationRequest* with *startActivityForResult*.

```
private void doAuthorization() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent authIntent = authService.getAuthorizationRequestIntent(authRequest);
    startActivityForResult(authIntent, INIT_AUTH);
}
```

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == INIT_AUTH) {
        AuthorizationResponse resp = AuthorizationResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

6 Capture the authorization redirect. After a user signs in with managed login, Amazon Cognito redirects them to the return URL. Configure a handler at the return URL for Android. Use a manifest placeholder for a custom URL scheme. If your return URL has an HTTPS URL schema, you can add an intent filter for *RedirectUriReceiverActivity* in *AndroidManifest.xml*.

```
android.defaultConfig.manifestPlaceholders = [
  'appAuthRedirectScheme': 'cognito.sample.android.demo'
]
```

7 Get the authorization response from the *fromIntent()* methods *AuthorizationResponse* or *AuthorizationException*.

```
public void onStart() {
    AuthorizationResponse response = AuthorizationResponse.fromIntent(getIntent());
    AuthorizationException ex = AuthorizationException.fromIntent(getIntent());
    if (response != null) {
        // authorization succeeded
    } else {
        // authorization failed
    }
}
```

8 Configure a handler for the *AuthorizationResponse* to exchange the authorization code for user pool tokens. Assemble a token-exchange request with *createTokenExchangeRequest* and implement *TokenResponseCallback()*. A successful exchange produces a *TokenResponse* object.

```
authService.performTokenRequest(
    authorizationResponse.createTokenExchangeRequest(),
    new AuthorizationService.TokenResponseCallback() {
        @Override
        public void onTokenRequestCompleted(TokenResponse response, AuthorizationException ex) {
            // use the response, such as authState.update(response, ex);
            if (response != null) {
                // exchange succeeded
            } else {
                // exchange failed
            }
        }
    });
```

9 Configure session-end behavior for users who sign out. Assemble an *EndSessionRequest* to your user pool logout endpoint.

```
EndSessionRequest endSessionRequest =
    new EndSessionRequest.Builder(cognitoServiceConfiguration)
        .setIdTokenHint(ID_TOKEN) // retrieve the ID_TOKEN from the TokenResponse
        .setPostLogoutRedirectUri("<logout uri>")
        .setAdditionalParameters(Map.of(
            "client_id", "3lsr6jpphlql4hb8n1ge6a8sjp",
            "logout_uri", "<logout uri>"))
        .build();
// Note: Your app's end-session redirect URI must be added to your Cognito app client Allowed sign out URLs.
```

10 Dispatch the *EndSessionRequest* with *startActivityForResult*.

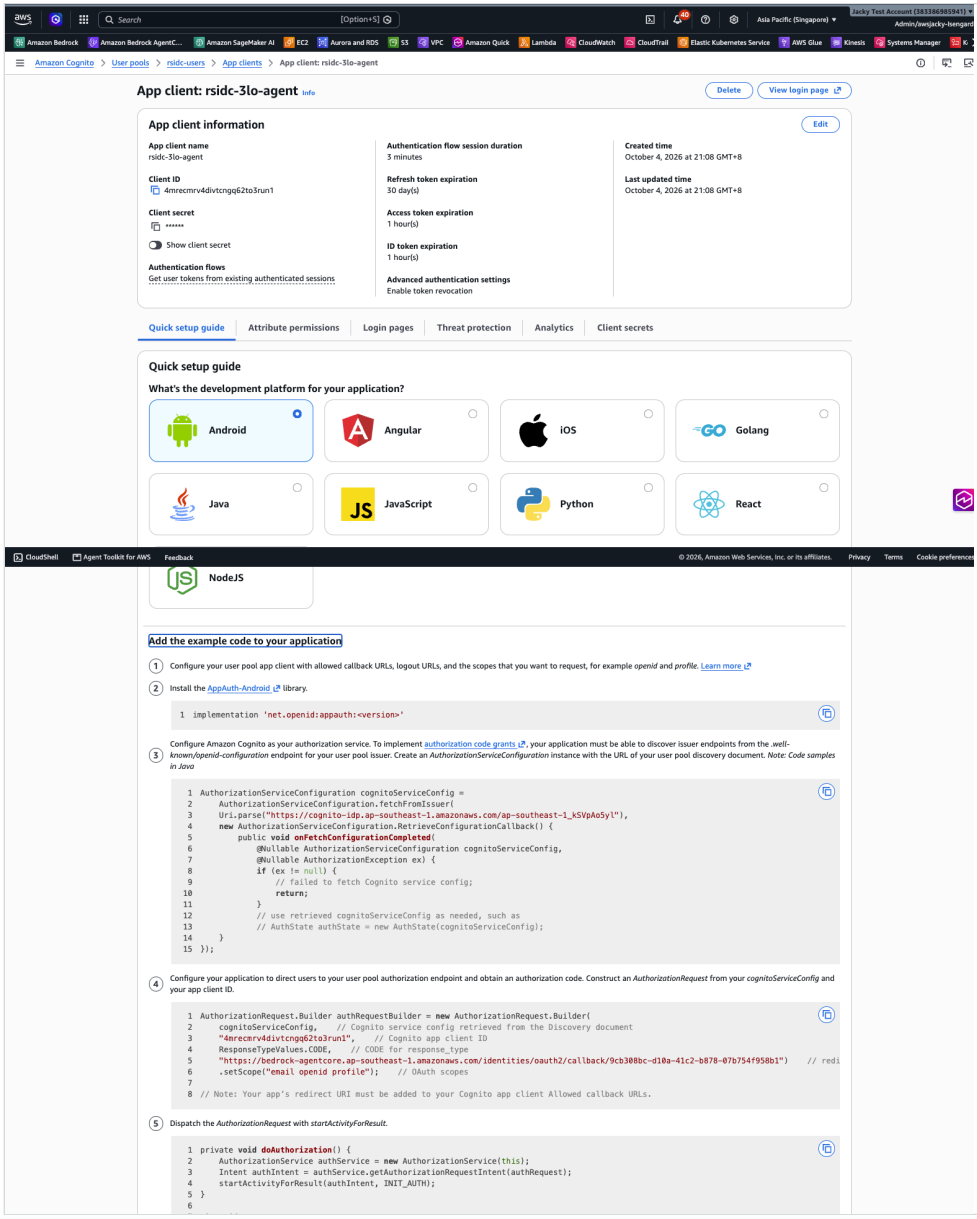
```
private void endSession() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent endSessionIntent = authService.getEndSessionRequestIntent(endSessionRequest);
    startActivityForResult(endSessionIntent, END_SESSION);
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == END_SESSION) {
        EndSessionResponse resp = EndSessionResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

part 2/2

Console: 3LO client: authorization code, callback = AgentCore Identity callback URL

URL at capture (2026-10-05 16:17:33 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/app-clients/4mrecmrv4divtcngg62to3run1/quick-setup-guide?region=ap-southeast-1



Add the example code to your application

1. Configure your user pool app client with allowed callback URLs, logout URLs, and the scopes that you want to request, for example *openid* and *profile*. Learn more

2. Install the AppAuth-Android library.

```
implementation 'net.openid:appauth:<version>'
```

3. Configure Amazon Cognito as your authorization service. To implement authorization code grants, your application must be able to discover issuer endpoints from the *.well-known/openid-configuration* endpoint for your user pool issuer. Create an *AuthorizationServiceConfiguration* instance with the URL of your user pool discovery document. *Note: Code samples in Java*

```
AuthorizationServiceConfiguration cognitoServiceConfig =
    AuthorizationServiceConfiguration.fetchFromIssuer(
    Uri.parse("https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_kSVpAo5yl"),
    new AuthorizationServiceConfiguration.RetrieveConfigurationCallback() {
        public void onFetchConfigurationCompleted(
            @Nullable AuthorizationServiceConfiguration cognitoServiceConfig,
            @Nullable AuthorizationException ex) {
            if (ex != null) {
                // failed to fetch Cognito service config;
                return;
            }
            // use retrieved cognitoServiceConfig as needed, such as
            // AuthState authState = new AuthState(cognitoServiceConfig);
    }
});
```

4. Configure your application to direct users to your user pool authorization endpoint and obtain an authorization code. Construct an *AuthorizationRequest* from your *cognitoServiceConfig* and your app client ID.

```
AuthorizationRequest.Builder authRequestBuilder = new AuthorizationRequest.Builder(
    cognitoServiceConfig,    // Cognito service config retrieved from the Discovery document
    "4mrecmrv4divtcngq62to3run1",    // Cognito app client ID
    ResponseTypeValues.CODE,    // CODE for response_type
    "https://bedrock-agentcore.ap-southeast-1.amazonaws.com/identities/oauth2/callback/9cb308bc-d10a-41c2-b878-07b754f958b1")    // redi
    .setScope("email openid profile");    // OAuth scopes

// Note: Your app's redirect URI must be added to your Cognito app client Allowed callback URLs.
```

5. Dispatch the *AuthorizationRequest* with *startActivityForResult*.

```
private void doAuthorization() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent authIntent = authService.getAuthorizationRequestIntent(authRequest);
    startActivityForResult(authIntent, INIT_AUTH);
}
```

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == INIT_AUTH) {
        AuthorizationResponse resp = AuthorizationResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

6 Capture the authorization redirect. After a user signs in with managed login, Amazon Cognito redirects them to the return URL. Configure a handler at the return URL for Android. Use a manifest placeholder for a custom URL scheme. If your return URL has an HTTPS URL schema, you can add an intent filter for *RedirectUriReceiverActivity* in *AndroidManifest.xml*.

```
android.defaultConfig.manifestPlaceholders = [
  'appAuthRedirectScheme': 'cognito.sample.android.demo'
]
```

7 Get the authorization response from the *fromIntent()* methods *AuthorizationResponse* or *AuthorizationException*.

```
public void onStart() {
    AuthorizationResponse response = AuthorizationResponse.fromIntent(getIntent());
    AuthorizationException ex = AuthorizationException.fromIntent(getIntent());
    if (response != null) {
        // authorization succeeded
    } else {
        // authorization failed
    }
}
```

8 Configure a handler for the *AuthorizationResponse* to exchange the authorization code for user pool tokens. Assemble a token-exchange request with *createTokenExchangeRequest* and implement *TokenResponseCallback()*. A successful exchange produces a *TokenResponse* object.

```
authService.performTokenRequest(
    authorizationResponse.createTokenExchangeRequest(),
    new AuthorizationService.TokenResponseCallback() {
        @Override
        public void onTokenRequestCompleted(TokenResponse response, AuthorizationException ex) {
            // use the response, such as authState.update(response, ex);
            if (response != null) {
                // exchange succeeded
            } else {
                // exchange failed
            }
        }
    });
```

9 Configure session-end behavior for users who sign out. Assemble an *EndSessionRequest* to your user pool logout endpoint.

```
EndSessionRequest endSessionRequest =
    new EndSessionRequest.Builder(cognitoServiceConfiguration)
        .setIdTokenHint(ID_TOKEN) // retrieve the ID_TOKEN from the TokenResponse
        .setPostLogoutRedirectUri("<logout uri>")
        .setAdditionalParameters(Map.of(
            "client_id", "4mrecmrv4divtcnqq62to3run1",
            "logout_uri", "<logout uri>"))
        .build();
// Note: Your app's end-session redirect URI must be added to your Cognito app client Allowed sign out URLs.
```

10 Dispatch the *EndSessionRequest* with *startActivityForResult*.

```
private void endSession() {
    AuthorizationService authService = new AuthorizationService(this);
    Intent endSessionIntent = authService.getEndSessionRequestIntent(endSessionRequest);
    startActivityForResult(endSessionIntent, END_SESSION);
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == END_SESSION) {
        EndSessionResponse resp = EndSessionResponse.fromIntent(data);
        AuthorizationException ex = AuthorizationException.fromIntent(data);
        // ... process the result
    } else {
        // ...
    }
}
```

part 2/2

Verification

Check	Command	Expected	Result
domain on managed login v2	`aws cognito-idp describe-user-pool-domain --region $RT_REGION --domain rsidc-383386985941 --query 'DomainDescription.[Status,ManagedLoginVersion]' --output text`	`ACTIVE\s+2`	**PASS**
2LO client = client_credentials only	`aws cognito-idp describe-user-pool-client --region $RT_REGION --user-pool-id $COGNITO_POOL_ID --client-id $C2LO_CLIENT_ID --query 'UserPoolClient.[AllowedOAuthFlows,AllowedOAuthScopes]' --output text`	`client_credentials\s+rsidc-reports/read`	**PASS**
3LO client callback = AgentCore Identity	`aws cognito-idp describe-user-pool-client --region $RT_REGION --user-pool-id $COGNITO_POOL_ID --client-id $C3LO_CLIENT_ID --query 'UserPoolClient.[AllowedOAuthFlows[0],CallbackURLs[0]]' --output text`	`code\s+https://bedrock-agentcore.ap-southeast-1.amazonaws.com/identities/oauth2/callback/`	**PASS**

▼ Actual output: domain on managed login v2

```
ACTIVE  2
```

▼ Actual output: 2LO client = client_credentials only

```
client_credentials
rsidc-reports/read
```

▼ Actual output: 3LO client callback = AgentCore Identity

```
code    https://bedrock-agentcore.ap-southeast-1.amazonaws.com/identities/oauth2/callback/9cb308bc-d10a-41c2-b878-07b754f958b1
```

Step 10: AgentCore Identity credential providers

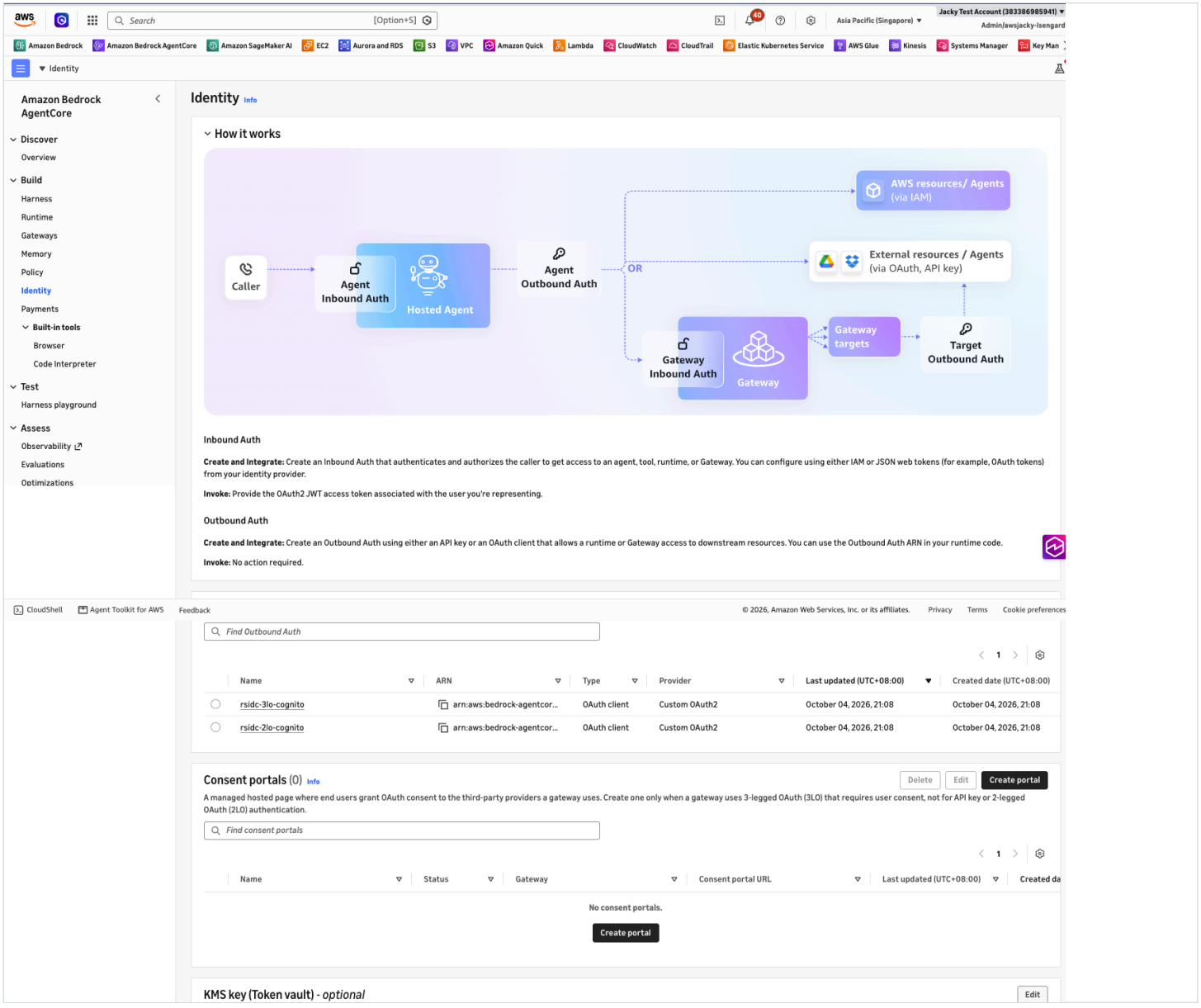
Two CustomOauth2 providers backed by Cognito discovery: `rsidc-2lo-cognito` (M2M) and `rsidc-3lo-cognito` (user federation). Client secrets are held by AgentCore Identity (Secrets Manager, never printed).

IaC / script: `infra/10-credential-providers.py`
Deploy command: `infra/run.sh agent/.venv/bin/python infra/10-credential-providers.py`

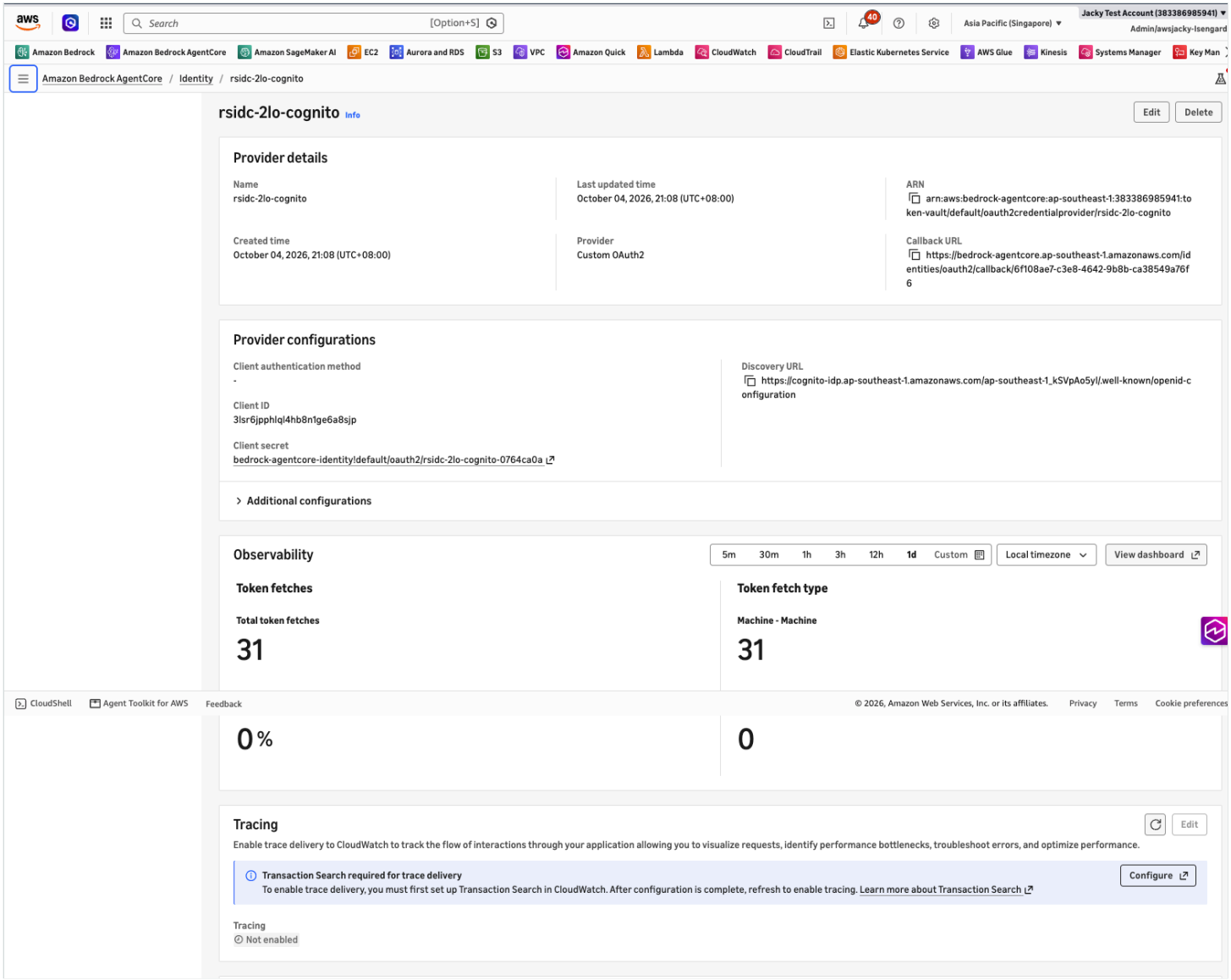
Deployed resources

Type	ID / ARN	Name
AWS::BedrockAgentCore::OAuth2CredentialProvider	`arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:token-vault/default/oauth2credentialprovider/rsidc-2lo-cognito`	rsidc-2lo-cognito
AWS::BedrockAgentCore::OAuth2CredentialProvider	`arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:token-vault/default/oauth2credentialprovider/rsidc-3lo-cognito`	rsidc-3lo-cognito

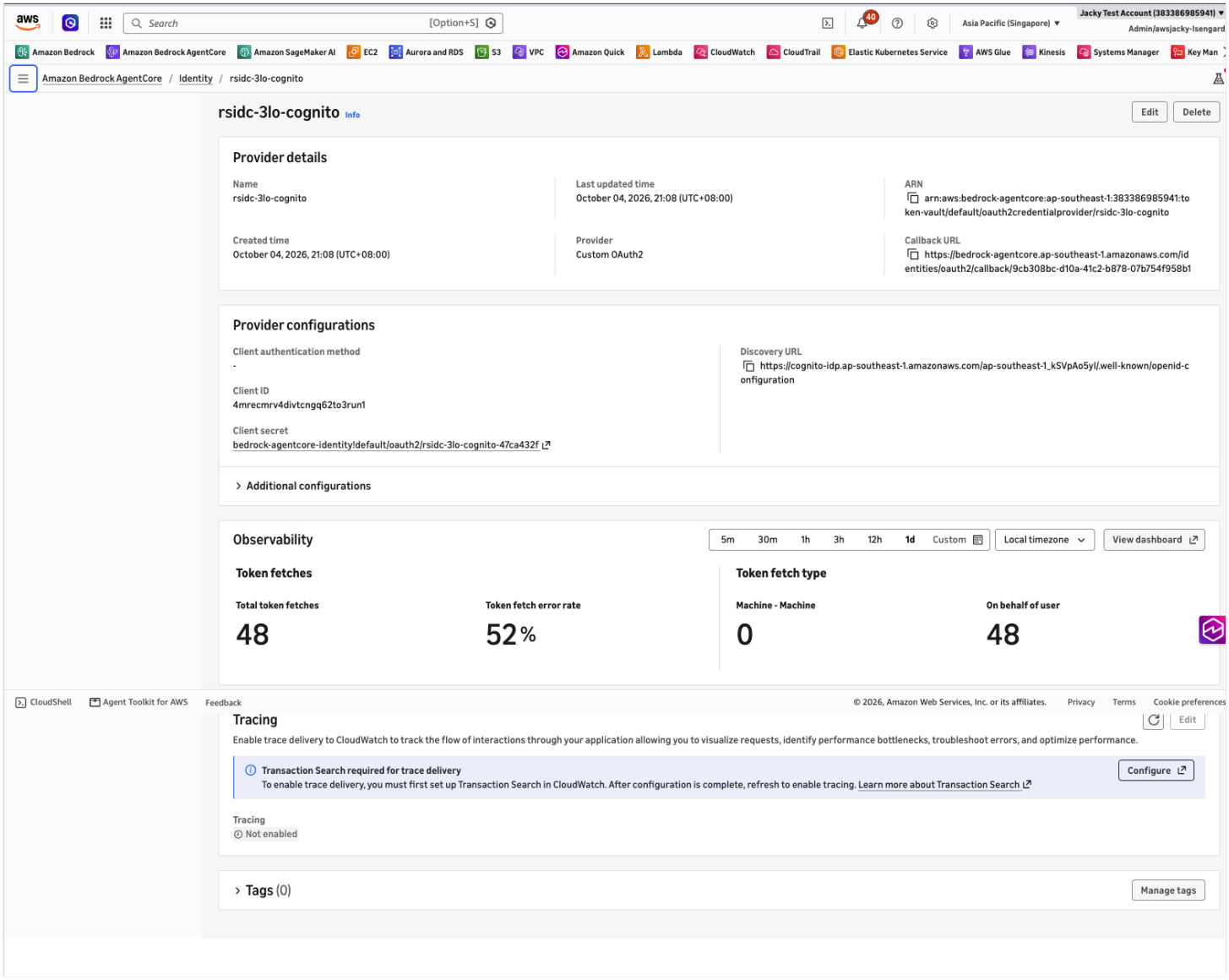
Console: AgentCore Identity → Outbound Auth: rsidc-2lo-cognito, rsidc-3lo-cognito
URL at capture (2026-10-05 16:17:41 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/identity?region=ap-southeast-1



Console: Credential provider rsidc-2lo-cognito (Custom OAuth2, M2M)
URL at capture (2026-10-05 16:17:49 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/identity/rsidc-2lo-cognito?region=ap-southeast-1



Console: Credential provider rsidc-3lo-cognito (Custom OAuth2, user federation)
URL at capture (2026-10-05 16:17:58 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/identity/rsidc-3lo-cognito?region=ap-southeast-1



Verification

Check	Command	Expected
2LO provider → 2LO client	`aws bedrock-agentcore-control get-oauth2-credential-provider --region $RT_REGION --name rsidc-2lo-cognito --query '[status,credentialProviderVendor,oauth2ProviderConfigOutput.customOauth2ProviderConfig.clientId]' --output text`	`READY\s+CustomOauth2\s+3lsr6jpphlql4hb8n1ge6a8sjp`
3LO provider → 3LO client	`aws bedrock-agentcore-control get-oauth2-credential-provider --region $RT_REGION --name rsidc-3lo-cognito --query '[status,oauth2ProviderConfigOutput.customOauth2ProviderConfig.clientId]' --output text`	`READY\s+4mrecmrv4divtcngq62to3run1`

▼ Actual output: 2LO provider → 2LO client

```
READY   CustomOauth2    3lsr6jpphlql4hb8n1ge6a8sjp
```

▼ Actual output: 3LO provider → 3LO client

```
READY   4mrecmrv4divtcngq62to3run1
```

Step 11: 2LO downstream: Gateway → Lambda → Redshift service user

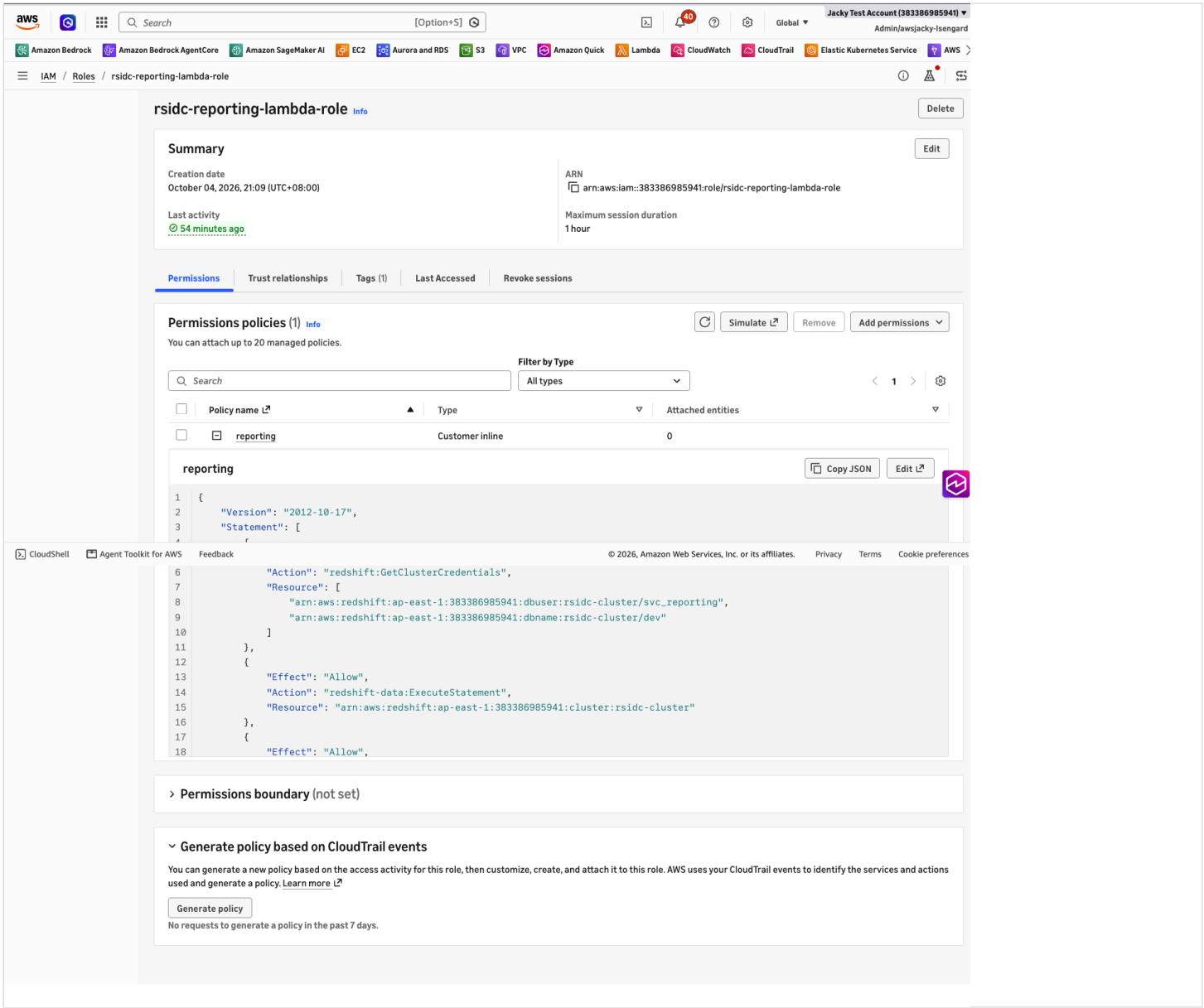
DB user `svc_reporting`; Lambda that queries Redshift through the Data API as that user; AgentCore Gateway (MCP) whose JWT authorizer accepts only the 2LO client; target `reporting` invoked with the Gateway IAM role.

IaC / script: `infra/11-2lo-gateway.sh + lambda/reporting/ + infra/sql/02-svc-reporting.sql`
Deploy command: `infra/run.sh ./infra/11-2lo-gateway.sh`

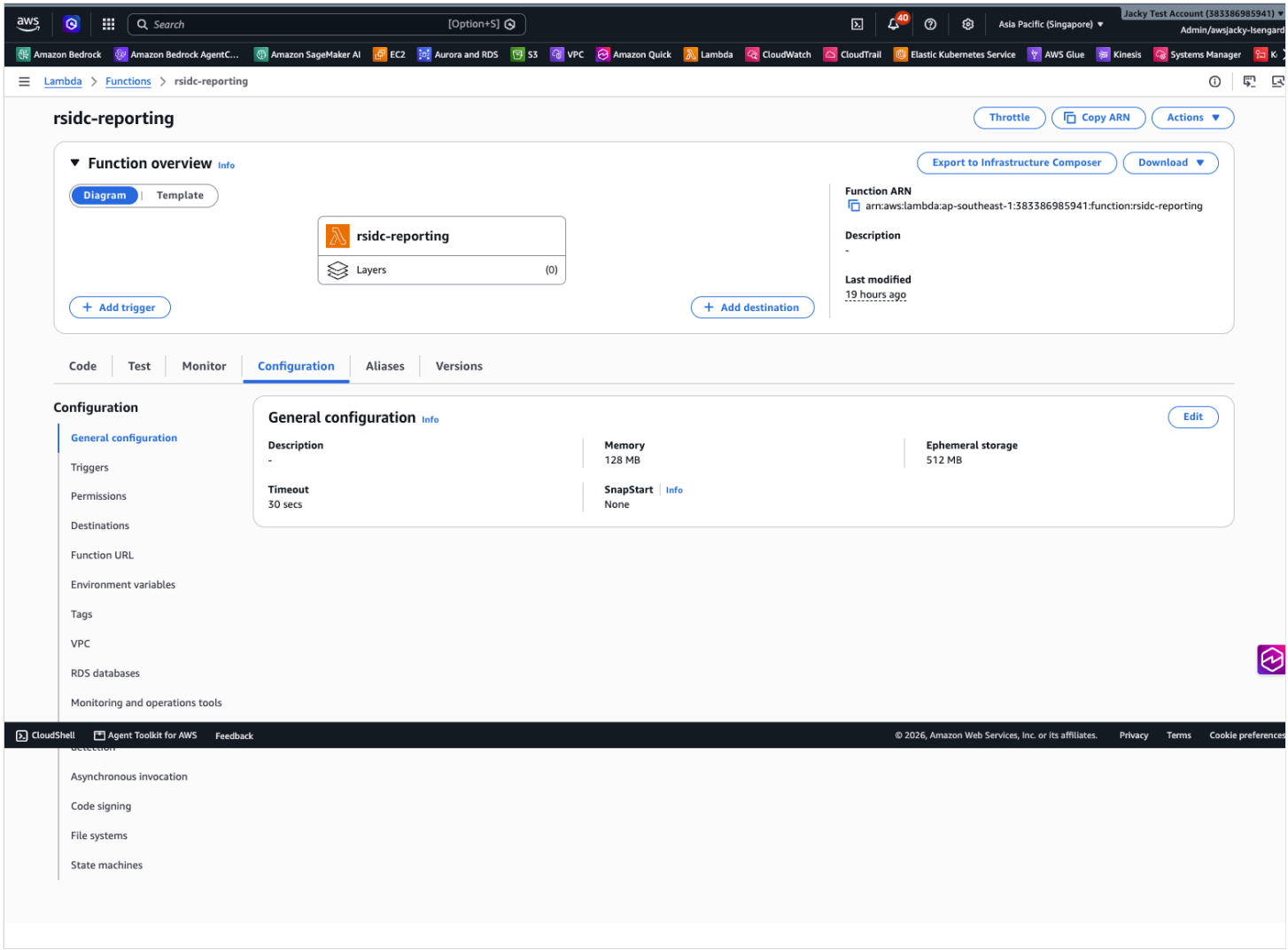
Deployed resources

Type	ID / ARN	Name
AWS::IAM::Role	`rsidc-reporting-lambda-role`	Data API as svc_reporting
AWS::Lambda::Function	`arn:aws:lambda:ap-southeast-1:383386985941:function:rsidc-reporting`	rsidc-reporting
AWS::IAM::Role	`rsidc-gateway-role`	invoke rsidc-reporting
AWS::BedrockAgentCore::Gateway	`rsidc-2lo-gateway-mhss1cesvt`	rsidc-2lo-gateway
AWS::BedrockAgentCore::GatewayTarget	`DW1PNIOIAE`	reporting (Lambda)
Redshift user	`svc_reporting`	SELECT sales.orders

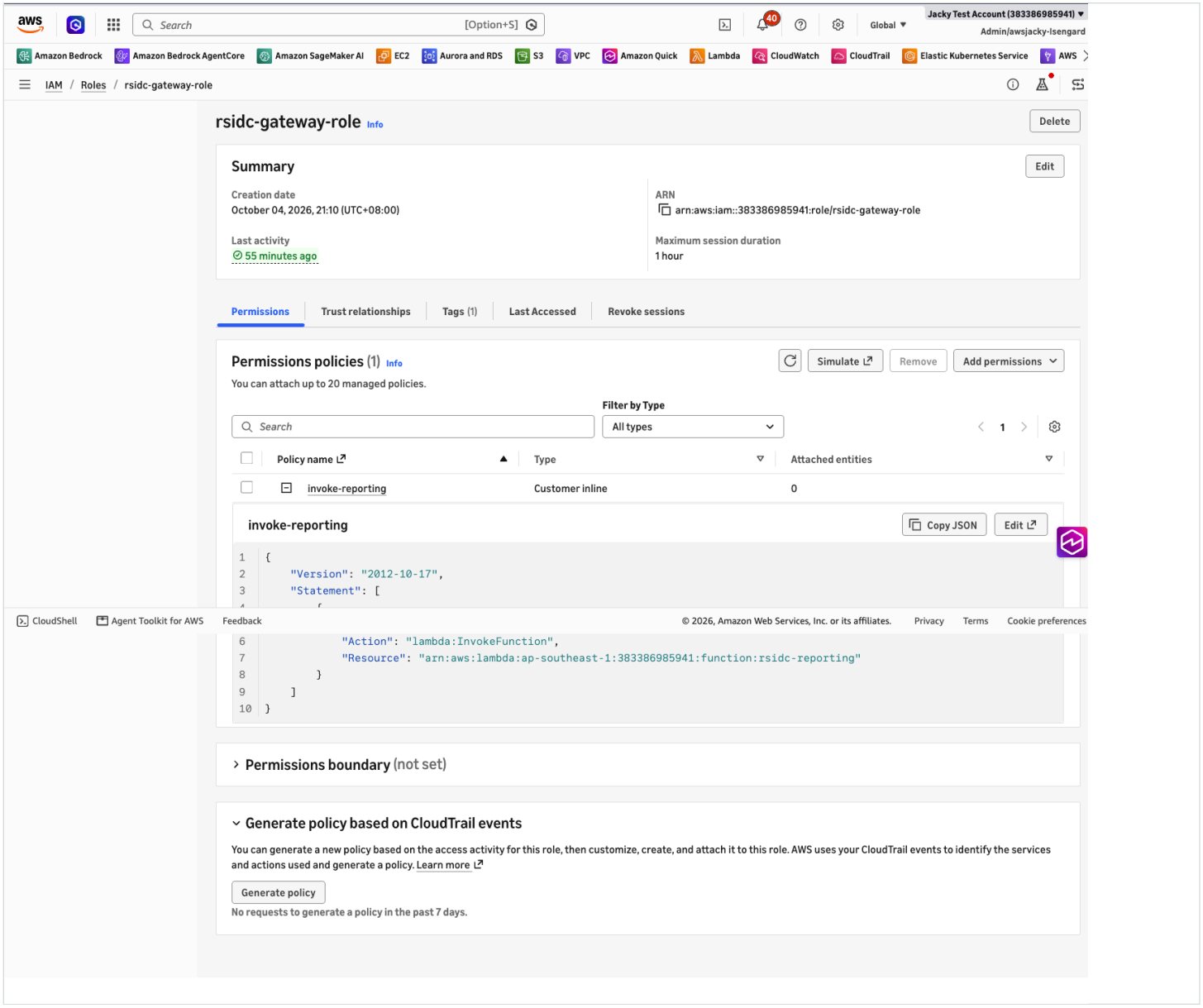
Console: Lambda role: GetClusterCredentials only for dbuser svc_reporting + Data API
URL at capture (2026-10-05 16:18:11 HKT): https://us-east-1.console.aws.amazon.com/iam/home#/roles/details/rsidc-reporting-lambda-role?section=permissions



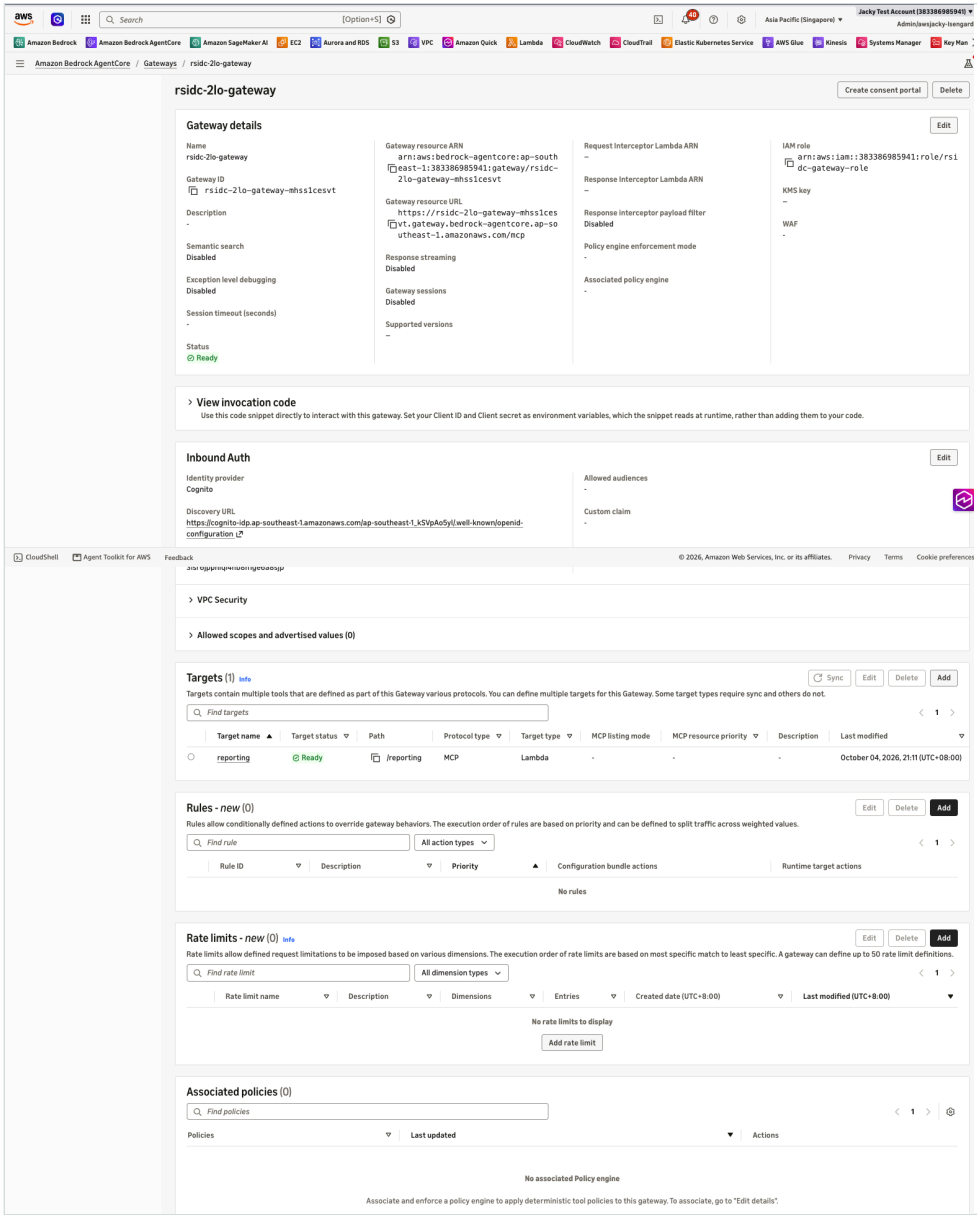
Console: Lambda rsidc-reporting (Gateway target; queries Redshift as svc_reporting)
URL at capture (2026-10-05 16:18:18 HKT): https://ap-southeast-1.console.aws.amazon.com/lambda/home?region=ap-southeast-1#/functions/rsidc-reporting?tab=configure



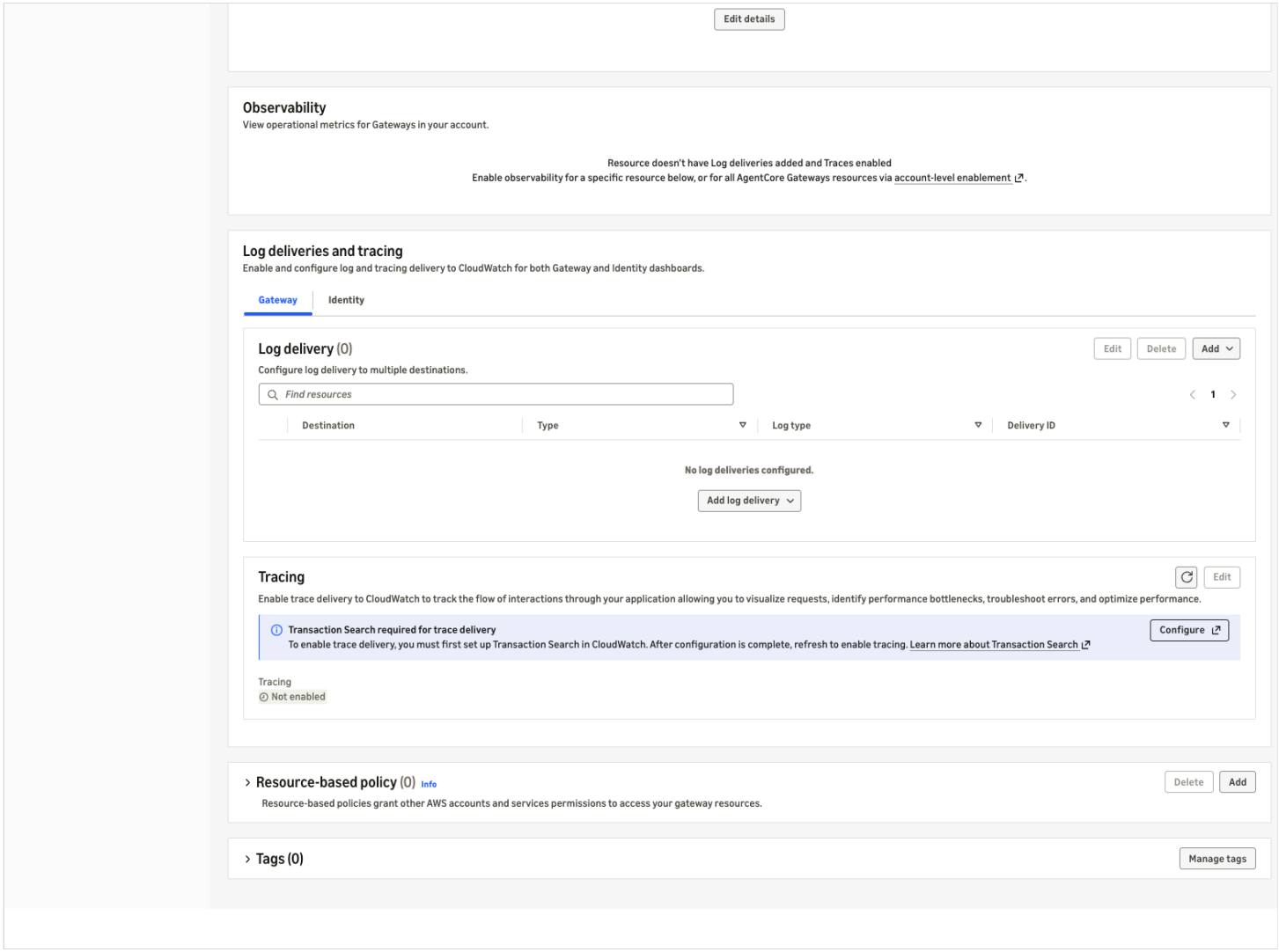
Console: Gateway role: lambda:InvokeFunction on rsidc-reporting only
URL at capture (2026-10-05 16:18:31 HKT): https://us-east-1.console.aws.amazon.com/iam/home#/roles/details/rsidc-gateway-role?section=permissions



Console: AgentCore Gateway rsidc-2lo-gateway (MCP), target reporting → Lambda
URL at capture (2026-10-05 16:18:47 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/toolsAndGateways/rsidc-2lo-gateway-mhss1cesvt


Search
[Option+S]
Asia Pacific (Singapore)
Jacky Test Account (383386985941)
Admin/awsjacky-lsengard
Amazon Bedrock
Amazon Bedrock AgentCore
Amazon SageMaker AI
EC2
Aurora and RDS
S3
VPC
Amazon Quick
Lambda
CloudWatch
CloudTrail
Elastic Kubernetes Service
AWS Glue
Kinesis
Systems Manager
Key Man
Amazon Bedrock AgentCore / Gateways / rsidc-2lo-gateway
rsidc-2lo-gateway
Create consent portal
Delete
Gateway details
Edit
Name
rsidc-2lo-gateway
Gateway ID
rsidc-2lo-gateway-mhss1cesvt
Description
-
Semantic search
Disabled
Exception level debugging
Disabled
Session timeout (seconds)
-
Status
Ready
Gateway resource ARN
arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:gateway/rsidc-2lo-gateway-mhss1cesvt
Gateway resource URL
https://rsidc-2lo-gateway-mhss1cesvt.gateway.bedrock-agentcore.ap-southeast-1.amazonaws.com/mcp
Response streaming
Disabled
Gateway sessions
Disabled
Supported versions
–
Request interceptor Lambda ARN
–
Response interceptor Lambda ARN
–
Response interceptor payload filter
Disabled
Policy engine enforcement mode
-
Associated policy engine
-
IAM role
arn:aws:iam::383386985941:role/rsidc-gateway-role
KMS key
–
WAF
-
View invocation code
Use this code snippet directly to interact with this gateway. Set your Client ID and Client secret as environment variables, which the snippet reads at runtime, rather than adding them to your code.
Inbound Auth
Edit
Identity provider
Cognito
Discovery URL
https://cognito-idp.ap-southeast-1.amazonaws.com/ap-southeast-1_kSVpAo5yl/.well-known/openid-configuration
Allowed audiences
-
Custom claim
-
CloudShell
Agent Toolkit for AWS
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
VPC Security
Allowed scopes and advertised values (0)
Targets (1) Info
Sync
Edit
Delete
Add
Targets contain multiple tools that are defined as part of this Gateway various protocols. You can define multiple targets for this Gateway. Some target types require sync and others do not.
Find targets
Target name
Target status
Path
Protocol type
Target type
MCP listing mode
MCP resource priority
Description
Last modified
reporting
Ready
/reporting
MCP
Lambda
October 04, 2026, 21:11 (UTC+08:00)
Rules - new (0)
Edit
Delete
Add
Rules allow conditionally defined actions to override gateway behaviors. The execution order of rules are based on priority and can be defined to split traffic across weighted values.
Find rule
All action types
Rule ID
Description
Priority
Configuration bundle actions
Runtime target actions
No rules
Rate limits - new (0) Info
Edit
Delete
Add
Rate limits allow defined request limitations to be imposed based on various dimensions. The execution order of rate limits are based on most specific match to least specific. A gateway can define up to 50 rate limit definitions.
Find rate limit
All dimension types
Rate limit name
Description
Dimensions
Entries
Created date (UTC+8:00)
Last modified (UTC+8:00)
No rate limits to display
Add rate limit
Associated policies (0)
Find policies
Policies
Last updated
Actions
No associated Policy engine
Associate and enforce a policy engine to apply deterministic tool policies to this gateway. To associate, go to "Edit details".

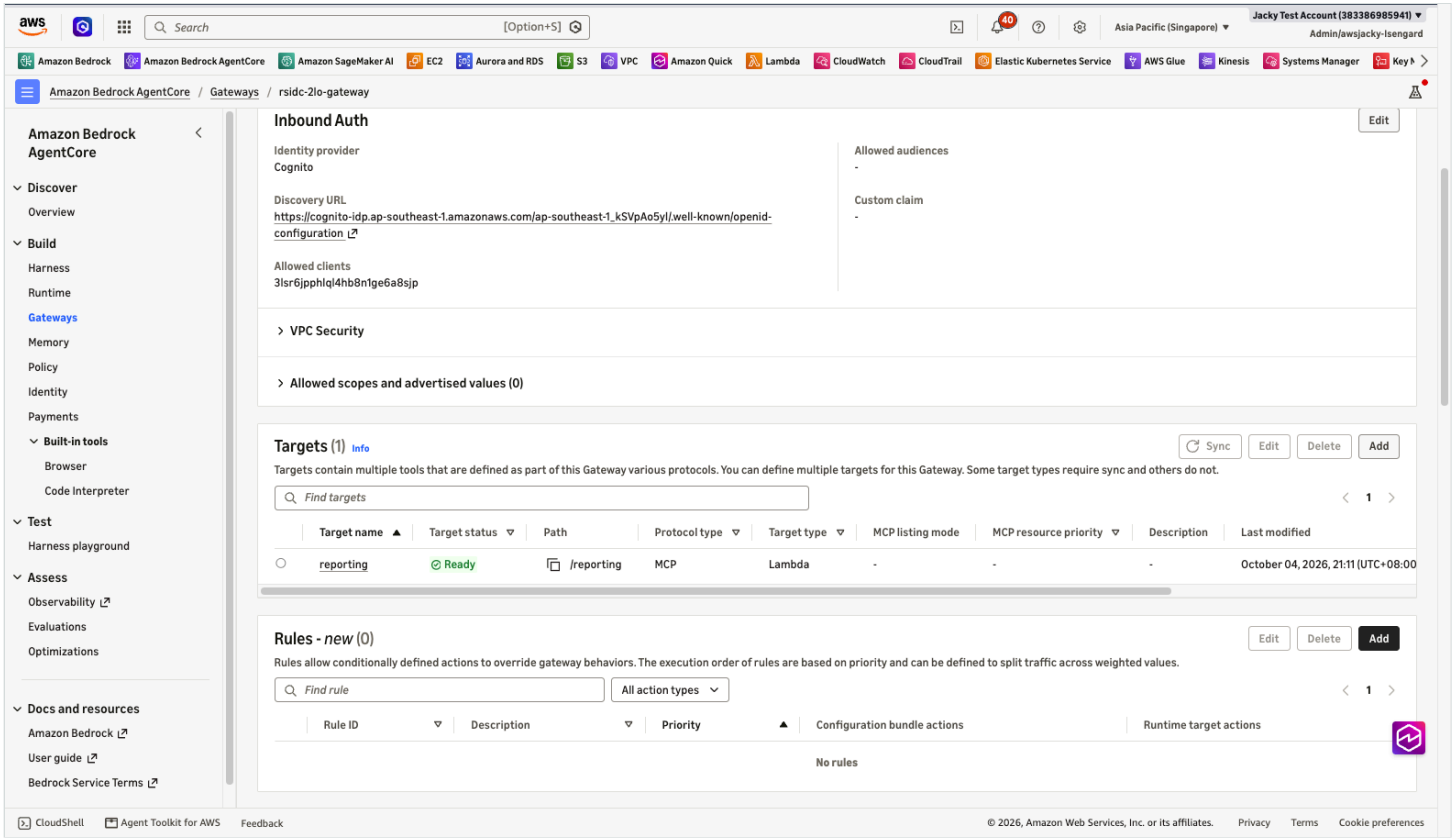
part 1/2



part 2/2

Console: Gateway Inbound Auth: Cognito, Allowed clients = 2LO client only

URL at capture (2026-10-05 16:19:05 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/toolsAndGateways/rsidc-2lo-gateway-mhss1cesvt



Verification

Check	Command	Expected	Result
gateway READY, allowedClients = 2LO client	`aws bedrock-agentcore-control get-gateway --region $RT_REGION --gateway-identifier $GW_ID --query '[status,protocolType,authorizerConfiguration.customJWTAuthorizer.allowedClients[0]]' --output text`	`READY\s+MCP\s+3lsr6jpphlql4hb8n1ge6a8sjp`	PASS
target reporting READY	`aws bedrock-agentcore-control list-gateway-targets --region $RT_REGION --gateway-identifier $GW_ID --query 'items[].[name,status]' --output text`	`reporting\s+READY`	PASS
Lambda may impersonate only svc_reporting	`aws iam get-role-policy --role-name rsidc-reporting-lambda-role --policy-name reporting --query 'PolicyDocument.Statement[0].Resource' --output text`	`dbuser:rsidc-cluster/svc_reporting`	PASS

▼ Actual output: gateway READY, allowedClients = 2LO client

```
READY   MCP     3lsr6jpphlql4hb8n1ge6a8sjp
```

▼ Actual output: target reporting READY

```
reporting       READY
```

▼ Actual output: Lambda may impersonate only svc_reporting

```
arn:aws:redshift:ap-east-1:383386985941:dbuser:rsidc-cluster/svc_reporting      arn:aws:redshift:ap-east-1:383386985941:dbname:rsidc-cluster/dev
```

Step 12: Separate OAuth agent runtime (2LO / 3LO)

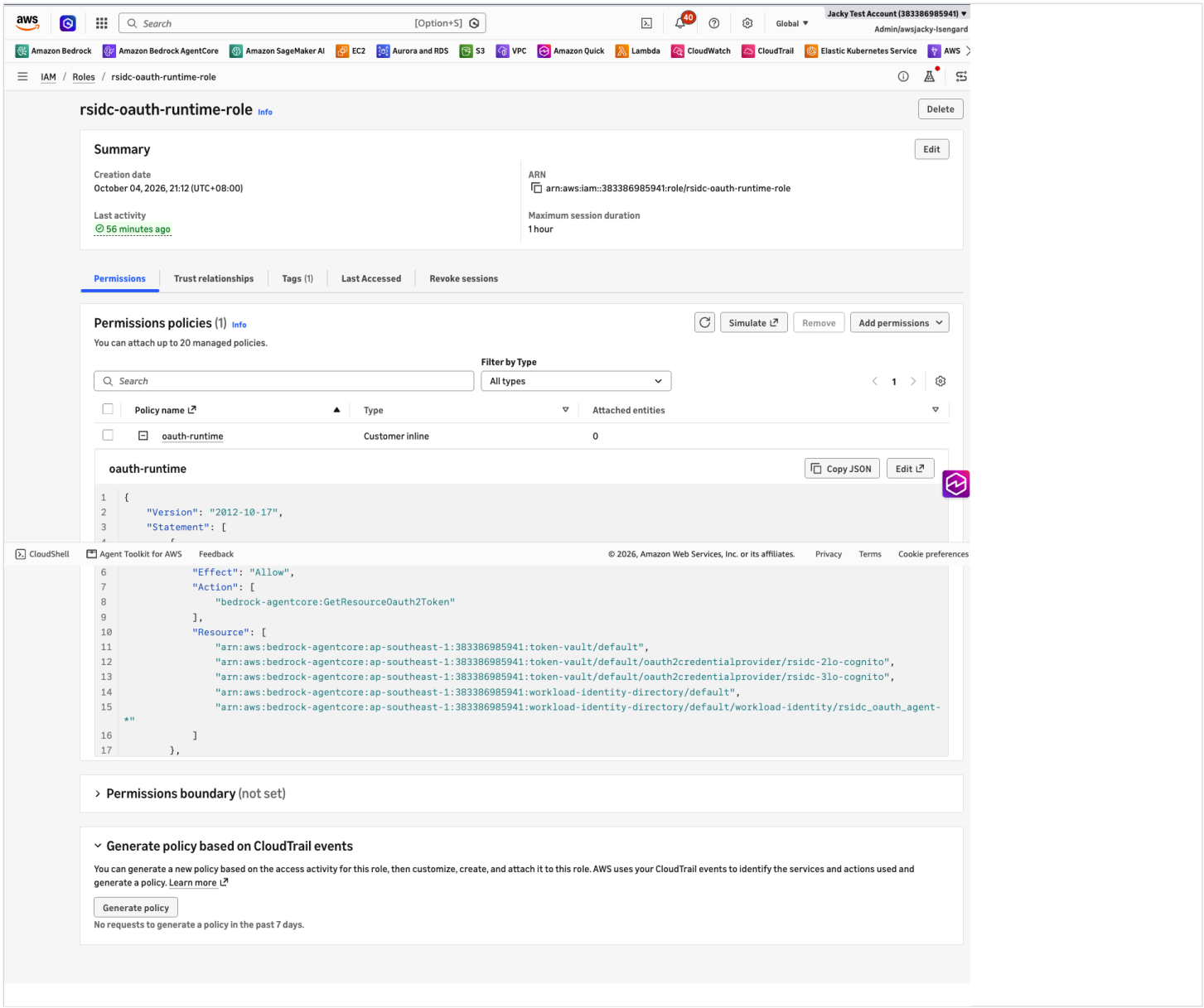
`rsidc_oauth_agent` with its own role. The role grants `GetResourceOauth2Token` on the token vault, both credential providers and its own workload identity, plus read access to the two provider client secrets; this is what lets the runtime use `rsidc-2lo-cognito` / `rsidc-3lo-cognito`. The workload identity allowlists the 3LO return URLs.

IaC / script: `infra/12-deploy-oauth-runtime.sh + agent_oauth/ + infra/policies/oauth-runtime-permissions.json`
Deploy command: `infra/run.sh ./infra/12-deploy-oauth-runtime.sh`

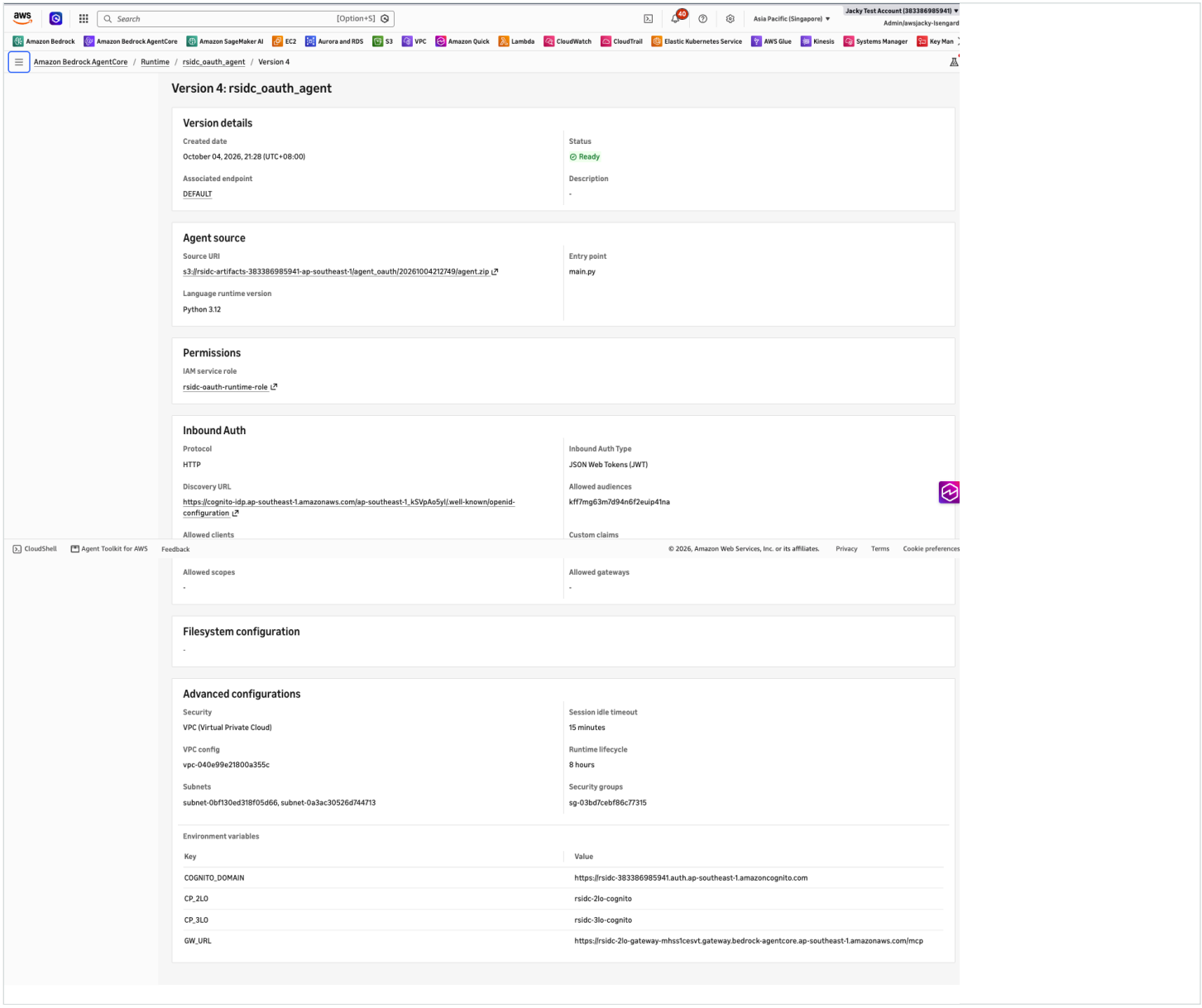
Deployed resources

Type	ID / ARN	Name
AWS::IAM::Role	`arn:aws:iam::383386985941:role/rsidc-oauth-runtime-role`	rsidc-oauth-runtime-role
AWS::BedrockAgentCore::Runtime	`arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:runtime/rsidc_oauth_agent-4lxrH9AmnT`	rsidc_oauth_agent (v4)
AWS::BedrockAgentCore::WorkloadIdentity	`rsidc_oauth_agent-4lxrH9AmnT`	return URLs :8765 (tests), :8081 (GUI)

Console: OAuth runtime role: GetResourceOauth2Token on token-vault + both providers + its workload identity
URL at capture (2026-10-05 16:19:17 HKT): https://us-east-1.console.aws.amazon.com/iam/home#/roles/details/rsidc-oauth-runtime-role?section=permissions



Console: OAuth runtime rsidc_oauth_agent v4: JWT inbound, VPC, env CP_2LO / CP_3LO / GW_URL
URL at capture (2026-10-05 16:19:25 HKT): https://ap-southeast-1.console.aws.amazon.com/bedrock-agentcore/agents/rsidc_oauth_agent-4lxrH9AmnT/version/4?region=ap-southeast-1



Verification

Check	Command	Expected
role grants GetResourceOauth2Token on the 2LO provider	`aws iam get-role-policy --role-name rsidc-oauth-runtime-role --policy-name oauth-runtime --query 'PolicyDocument.Statement[?Sid==`OutboundOAuthTokens`].Resource[]' --output text`	`oauth2credentialprovider/rsidc-2lo-cognito`
runtime READY with provider names in env	`aws bedrock-agentcore-control get-agent-runtime --region $RT_REGION --agent-runtime-id $OAUTH_RT_ID --query '[status,environmentVariables.CP_2LO,environmentVariables.CP_3LO]' --output text`	`READY\s+rsidc-2lo-cognito\s+rsidc-3lo-cognito`
workload identity return URLs	`aws bedrock-agentcore-control get-workload-identity --region $RT_REGION --name $OAUTH_WI_NAME --query allowedResourceOauth2ReturnUrls --output text`	`localhost:8765/callback.*localhost:8081/3lo/callback|localhost:8081`

▼ Actual output: role grants GetResourceOauth2Token on the 2LO provider

```
arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:token-vault/default	arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:token-vault/default/oauth2credentialprovider/rsidc-2lo-cognito	arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:token-vault/default/oauth2credentialprovider/rsidc-3lo-cognito	arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:workload-identity-directory/default	arn:aws:bedrock-agentcore:ap-southeast-1:383386985941:workload-identity-directory/default/workload-identity/rsidc_oauth_agent-*
```

▼ Actual output: runtime READY with provider names in env

```
READY	rsidc-2lo-cognito	rsidc-3lo-cognito
```

▼ Actual output: workload identity return URLs

```
http://localhost:8765/callback	http://localhost:8081/3lo/callback
```

Step 13: Browser sign-in for the local web GUIs

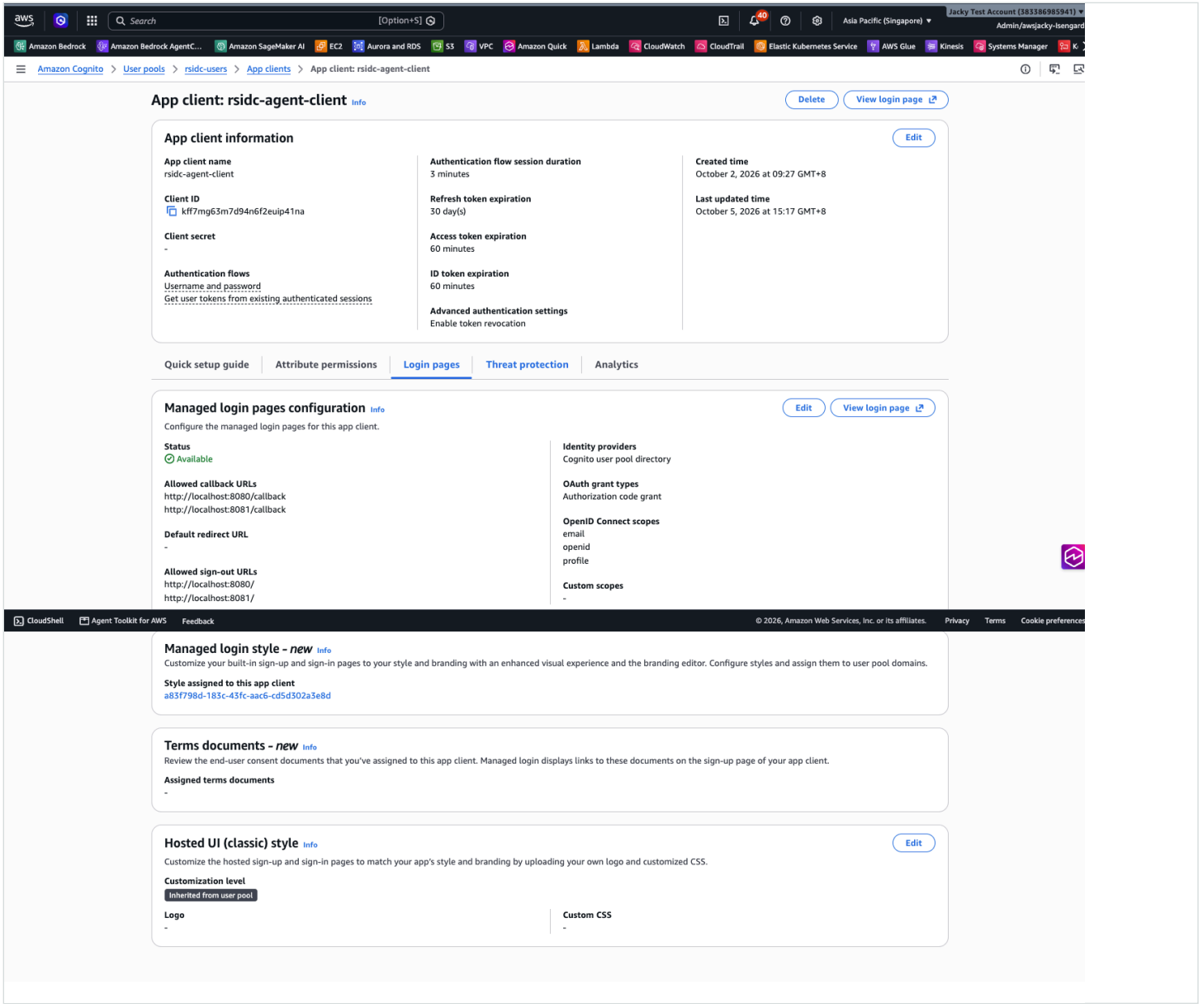
Adds managed login (authorization code + PKCE) and the callbacks `http://localhost:8080/callback` (OBO GUI) and `http://localhost:8081/callback` (2LO/3LO GUI) to the existing OBO client. Only OAuth fields change; the audience stays the same.

IaC / script: `infra/13-web-login-client.py + webchat/`
Deploy command: `infra/run.sh agent/.venv/bin/python infra/13-web-login-client.py`

Deployed resources

Type	ID / ARN	Name
AWS::Cognito::UserPoolClient (update)	`kff7mg63m7d94n6f2euip41na`	code flow + localhost callbacks

Console: OBO client managed login: callbacks http://localhost:8080/callback and :8081/callback (web GUIs)
URL at capture (2026-10-05 16:19:38 HKT): https://ap-southeast-1.console.aws.amazon.com/cognito/v2/idp/user-pools/ap-southeast-1_kSVpAo5yl/applications/app-clients/kff7mg63m7d94n6f2euip41na/login-pages?region=ap-southeast-1



Verification

Check	Command	Expected
code flow + both localhost callbacks; password auth kept	`aws cognito-idp describe-user-pool-client --region $RT_REGION --user-pool-id $COGNITO_POOL_ID --client-id $COGNITO_CLIENT_ID --query 'UserPoolClient.[AllowedOAuthFlows[0],join(`,`,CallbackURLs),join(`,`,ExplicitAuthFlows)]' --output text`	`code\s+http://localhost:8080/callback,http://localhost:8081/callback\s+`

▼ Actual output: code flow + both localhost callbacks; password auth kept

```
code    http://localhost:8080/callback,http://localhost:8081/callback   ALLOW_REFRESH_TOKEN_AUTH,ALLOW_USER_PASSWORD_AUTH
```

Findings

1. OBO enforces IdC per-user access in Redshift (16/16 matrix). The runtime role has no Redshift permission; every session runs as `AWSIDC:<user>`.
2. External JWTs are single-use at IdC; the agent exchanges once per sign-in (history/001).
3. An Allow-only IdC app ActorPolicy did not stop same-account admins; fixed with an explicit Deny (history/002, verified in step 7).
4. 2LO bypasses per-user controls by design: every user, including one with no grants, saw all rows with PII via `svc_reporting` (step 11).
5. 3LO: Token Vault isolation and session binding work; a shared IdP browser session can store another user's token, mitigated by the agent identity guard (history/003). `prompt=login` via customParameters stored no token (observed).
6. Console pitfalls (recorded in the aws-deploy-evidence skill): IdC tabs don't change the URL; Gateway deep links redirect to home; IAM inline policies must be expanded; full-page shots can hide a field behind the sticky footer.

Teardown

`infra/run.sh ./infra/99-teardown.sh` (asks for `delete rsidc`) deletes everything above in reverse order, including the Redshift cluster **without a final snapshot**. The pre-existing IAM Identity Center instance, users and groups are not touched. Running cost while deployed ≈ USD 0.72/hour.